Tokenization

Wei Xu

(Some slides from Tarek Naous, Kartik Goyal, Tatsunori Hashimoto)

(Recap) LLaMA

- Transformer variations that have been used in different LLMs
- Pre-normalization layer using RMSNorm
- SwiGLU activation function — combines Swish and Gated Linear Unit (GLU)
- Rotary positional embeddings (RoPE)
- AdamW Optimizer

More Recently

2025/01

deepseek

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI

research@deepseek.com

2025/03

Google DeepMind 2025-03-12

Gemma 3 Technical Report

Gemma Team, Google DeepMind[1]

We introduce Gemma 3, a multimodal addition to the Gemma family of lightweight open models, ranging in scale from 1 to 27 billion parameters. This version introduces vision understanding abilities, a wider coverage of languages and longer context – at least 128K tokens. We also change the architecture of the model to reduce the KV-cache memory that tends to explode with long context. This is achieved by increasing the ratio of local to global attention layers, and keeping the span on local attention short. The Gemma 3 models are trained with distillation and achieve superior performance to Gemma 2 for both pre-trained and instruction finetuned versions. In particular, our novel post-training recipe significantly improves the math, chat, instruction-following and multilingual abilities, making Gemma3-4B-IT competitive with Gemma2-27B-IT and Gemma3-27B-IT comparable to Gemini-1.5-Pro across benchmarks. We release all our models to the community.

2025/05

Qwen 2025-05-15

Qwen3 Technical Report

Qwen Team

https://huggingface.co/Qwen
https://modelscope.cn/organization/qwen
https://github.com/QwenLM/Qwen3

Abstract

In this work, we present Qwen3, the latest version of the Qwen model family. Qwen3 comprises a series of large language models (LLMs) designed to advance performance, efficiency, and multilingual capabilities. The Qwen3 series includes models of both dense and Mixture-of-Expert (MoE) architectures, with parameter scales ranging from 0.6 to 235 billion. A key innovation in Qwen3 is the integration of thinking mode (for complex, multi-step reasoning) and non-thinking mode (for rapid, context-driven responses) into a unified framework. This eliminates the need to switch between different models——such as chat-optimized models (e.g., GPT-4o) and dedicated reasoning models (e.g., QwQ-32B)——and enables dynamic mode switching based on user queries or chat templates. Meanwhile, Qwen3 introduces a thinking budget mechanism, allowing users to allocate computational resources adaptively during inference, thereby balancing latency and performance based on task complexity. Moreover, by leveraging the knowledge from the flagship models, we significantly reduce the computational resources required to build smaller-scale models, while ensuring their highly competitive performance. Empirical evaluations demonstrate that Qwen3 achieves state-of-the-art results across diverse benchmarks, including tasks in code generation, mathematical reasoning, agent tasks, etc., competitive against larger MoE models and proprietary models. Compared to its predecessor Qwen2.5, Qwen3 expands multilingual support from 29 to 119 languages and dialects, enhancing global accessibility through improved cross-lingual understanding and generation capabilities. To facilitate reproducibility and community-driven research and development, all Qwen3 models are publicly accessible under Apache 2.0.

What are being used?

- An evolving field. There are many architectural variations.
- A lot of empirical analysis is going into identifying best practices.

Aa Name	Has pa...	Link	# Year	Tokenizer type	# Vocab count	Norm	Parallel Layer	Pre-norm	Position embedding	Activations	MoE	# MLP factor	# num_layers
Original transformer	Yes	arxiv.org/abs....03762	2017	BPE	37000	LayerNorm	Serial	☐	Sine	ReLU	☐	4	6
GPT	Yes	cdn.openai.com/res...er.pdf	2018	BPE	40257	LayerNorm	Serial	☐	Absolute	GeLU	☐	4	12
GPT2	Yes	cdn.openai.com/bet...rs.pdf	2019	BPE	50257	LayerNorm	Serial	☑	Sine	GeLU	☐	4	48
T5 (11B)	Yes	arxiv.org/abs....10683	2019	SentencePiece	32128	RMSNorm	Serial	☑	Relative	ReLU	☐	64	24
GPT3 (175B)	Yes	arxiv.org/abs....14165	2020	BPE	50257	LayerNorm	Serial	☑	Sine	GeLU	☐	4	96
mT5	Yes	arxiv.org/abs....11934	2020	SentencePiece	250000	RMSNorm	Serial	☑	Relative	GeGLU	☐	2.5	24
T5 (XXL 11B) v1.1	Kind of	github.com/goo...d#t511	2020	SentencePiece	32128	RMSNorm	Serial	☑	Relative	GeGLU	☐	2.5	24
Gopher (280B)	Yes	arxiv.org/abs....11446	2021	SentencePiece	32000	RMSNorm	Serial	☑	Relative	ReLU	☐	4	80
Anthropic LM (not claude)	Yes	arxiv.org/abs....00861	2021	BPE	65536			☑			☐	4	64
LaMDA	Yes	arxiv.org/abs....08239	2021	BPE	32000			☑	Relative	GeGLU	☐	8	64
GPTJ	Kind of	huggingface.co/Ele...t-j-6b	2021	BPE	50257	LayerNorm	Parallel	☑	RoPE	GeLU	☐		28
Chinchilla	Yes	arxiv.org/abs....15556	2022	SentencePiece	32000	RMSNorm	Serial	☑	Relative	ReLU	☐	4	80
PaLM (540B)	Yes	arxiv.org/abs....02311	2022	SentencePiece	256000	RMSNorm	Parallel	☑	RoPE	SwiGLU	☐	4	118
OPT (175B)	Yes	arxiv.org/abs....01068	2022	BPE	50272	LayerNorm	Serial	☑	Absolute	ReLU	☐	4	96
BLOOM (175B)	Yes	arxiv.org/abs....05100	2022	BPE	250680	LayerNorm	Serial	☑	AliBi	GeLU	☐	4	70
GPT-NeoX	Yes	arxiv.org/pdf...45.pdf	2022	BPE	50257	LayerNorm	Parallel	☑	RoPE	GeLU	☐	4	44
GPT4 OPEN	Ad	arxiv.org/abs....08774	2023	BPE	100000			☐			☐		
LLaMA (65B)	Yes	arxiv.org/abs....13971	2023	BPE	32000	RMSNorm	Serial	☑	RoPE	SwiGLU	☐	2.6875	80
LLaMA2 (70B)	Yes	arxiv.org/abs....09288	2023	BPE	32000	RMSNorm	Serial	☑	RoPE	SwiGLU	☐	3.5	80
Mistral (7B)	Yes	arxiv.org/abs....06825	2023	BPE	32000	RMSNorm	Serial	☑	RoPE	SwiGLU	☐	3.5	32

Credit: Tatsu Hashimoto

What are being used? Converging?

Name	Year	Norm	Parallel Layer	Pre-norm	Position embedding	Activations
Original transformer	2017	LayerNorm	Serial	☐	Sine	ReLU
GPT	2018	LayerNorm	Serial	☐	Absolute	GeLU
T5 (11B)	2019	RMSNorm	Serial	☑	Relative	ReLU
GPT2	2019	LayerNorm	Serial	☑	Absolute	GeLU
T5 (XXL 11B) v1.1	2020	RMSNorm	Serial	☑	Relative	GeGLU
mT5	2020	RMSNorm	Serial	☑	Relative	GeGLU
GPT3 (175B)	2020	LayerNorm	Serial	☑	Absolute	GeLU
GPTJ	2021	LayerNorm	Parallel	☑	RoPE	GeLU
LaMDA	2021			☑	Relative	GeGLU
Anthropic LM (not claude)	2021			☑		
Gopher (280B)	2021	RMSNorm	Serial	☑	Relative	ReLU
GPT-NeoX	2022	LayerNorm	Parallel	☑	RoPE	GeLU
BLOOM (175B)	2022	LayerNorm	Serial	☑	AliBi	GeLU
OPT (175B)	2022	LayerNorm	Serial	☑	Absolute	ReLU
PaLM (540B)	2022	RMSNorm	Parallel	☑	RoPE	SwiGLU
Chinchilla	2022	RMSNorm	Serial	☑	Relative	ReLU
Mistral (7B)	2023	RMSNorm	Serial	☑	RoPE	SwiGLU
LLaMA2 (70B)	2023	RMSNorm	Serial	☑	RoPE	SwiGLU
LLaMA (65B)	2023	RMSNorm	Serial	☑	RoPE	SwiGLU
GPT4	2023			☐		
Baichuan 2	2023	RMSNorm	Serial	☑	AliBi	SwiGLU
Olmo 2	2024	RMSNorm	Serial	☐	RoPE	SwiGLU
Gemma 2 (27B)	2024	RMSNorm	Serial	☑	RoPE	GeGLU
Nemotron-4 (340B)	2024	LayerNorm	Serial	☑	RoPE	SqRelu
Qwen 2 (72b) - same for 2.5	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Falcon 2 11B	2024	LayerNorm	Parallel	☑	RoPE	GeLU
Phi3 (small) - same for phi4	2024	RMSNorm	Serial	☑	RoPE	GeGLU
Llama 3 (70B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Reka Flash	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Command R+	2024	LayerNorm	Parallel	☑	RoPE	SwiGLU
OLMo	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Qwen (14B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
DeepSeek (67B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Yi (34B)	2024	RMSNorm	Serial	☑	RoPE	SwiGLU
Mixtral of Experts	2024			☐		
Command A	2025	LayerNorm	Parallel	☑	Hybrid (RoPE+NoPE)	SwiGLU
Gemma 3	2025	RMSNorm	Serial	☐	RoPE	GeGLU
SmolLM2 (1.7B)	2025	RMSNorm	Serial	☑	RoPE	SwiGLU

- Pre-norm vs. Post-norm:
  all pre-norm, except Gemma uses both
- LayerNorm vs. RMSNorm:
  mostly RMSNorm
- Activation functions:
  all use some sorts of gating *GLU
- Position Embeddings:
  all RoPE, except Command A is hybrid

Image Credit: Tatsu Hashimoto

Tokenization

Tokenization

Monolingual models (30-50k vocab)

Model	Token count
Original transformer	37000
GPT	40257
GPT2/3	50257
T5/T5v1.1	32128
LLaMA	32000

Multilingual / Production Systems (100-250k vocab)

Model	Token count
mT5	250000
PaLM	256000
GPT4	100276
BLOOM	250680
DeepSeek	100000
Qwen 15B	152064
Yi	64000

Credit: Tatsu Hashimoto

Tokenization

- The non-google world uses BPE. Google uses the SentencePiece library, which (sometimes) refers to a non-BPE subword tokenizer

Model	Tokenizer
Original transformer	BPE
GPT 1/2/3	BPE
T5 / mT5 / T5v1.1	SentencePiece (Unigram)
Gopher/Chinchilla	SentencePiece (??)
PaLM	SentencePiece (??)
LLaMA	BPE

Credit: Tatsu Hashimoto

Subwords

gpt-4o

Token count
155

The Taj Mahal (/ˌtɑːdʒ məˈhɑːl, ˌtɑːʒ –/ TAHJ mə-HAH L, TAHZH –; Hindustani: [taːdʒ ˈmɛɦ(ɛ)l]; lit.'Crown of the Palace') is an ivory-white marble mausoleum on the right bank of the river Yamuna in Agra, Uttar Pradesh, India. It was commissioned in 1631 by the fifth Mughal emperor, Shah Jahan (r.1628–1658), to house the tomb of his beloved wife, Mumtaz Mahal; it also houses the tomb of Shah Jahan himself.

976, 87935, 162826, 96416, 135, 234, 83, 123874, 135, 238, 67, 134, 240, 31773, 135, 230, 71, 123874, 135, 238, 75, 11, 146774, 234, 83, 123874, 135, 238, 134, 240, 533, 14, 353, 29553, 41, 31773, 12, 12686, 22144, 11, 353, 29553, 127516, 533, 140605, 26, 63926, 570, 3048, 25, 723, 1524, 135, 238, 67, 134, 240, 146774, 230, 76, 14887, 133, 99, 7, 14887, 8, 75, 11464, 11980, 13, 29106, 194117, 7352, 328, 290, 39569, 1542, 382, 448, 131584, 25280, 60584, 148587, 169818, 402, 290, 1849, 6922, 328, 290, 20608, 53089, 4024, 306, 5582, 614, 11, 151598, 63318, 11, 8405, 13, 1225, 673, 73473, 306, 220, 17666, 16, 656, 290, 29598, 65480, 5512, 103767, 11, 65850, 643, 10712, 350, 81, 13, 29106, 18694, 23, 1585, 16461, 23, 936, 316, 4276, 290, 40089, 328, 1232, 37497, 11527, 11, 87718, 83, 1071, 162826, 26, 480, 1217, 20327, 290, 40089, 328, 65850, 643, 10712, 11166, 13, 220

gpt-4o

Token count
80

ताजमहल (उर्दू: تاج محل) भारत के उत्तर प्रदेश राज्य के आगरा शहर में स्थित एक विश्व धरोहर मक़बरा , और विश्व के ७ अजूबों में से एक है। इसका निर्माण १७वीं सदी में मुग़ल सम्राट शाहजहाँ ने अपनी पत्नी मुमताज़ महल की याद में करवाया था।

1329, 8353, 125948, 1455, 350, 16636, 52183, 2796, 25, 174780, 80625, 8, 29292, 2329, 64915, 45359, 47550, 2329, 53510, 21123, 59644, 3342, 74202, 6298, 41841, 13961, 23547, 15524, 147113, 5992, 3188, 21123, 1366, 5034, 41841, 2329, 220, 30961, 183451, 105925, 3824, 3342, 4291, 6298, 2487, 1670, 71751, 58826, 220, 170984, 152701, 1263, 25589, 3342, 34285, 2321, 5992, 1455, 7434, 2062, 17332, 137470, 74742, 14556, 6330, 27981, 111591, 1559, 20762, 1329, 196863, 13861, 1455, 4042, 61473, 3342, 151779, 18546, 14521, 1670

https://tiktokenizer.vercel.app/

Rare/Unknown Words

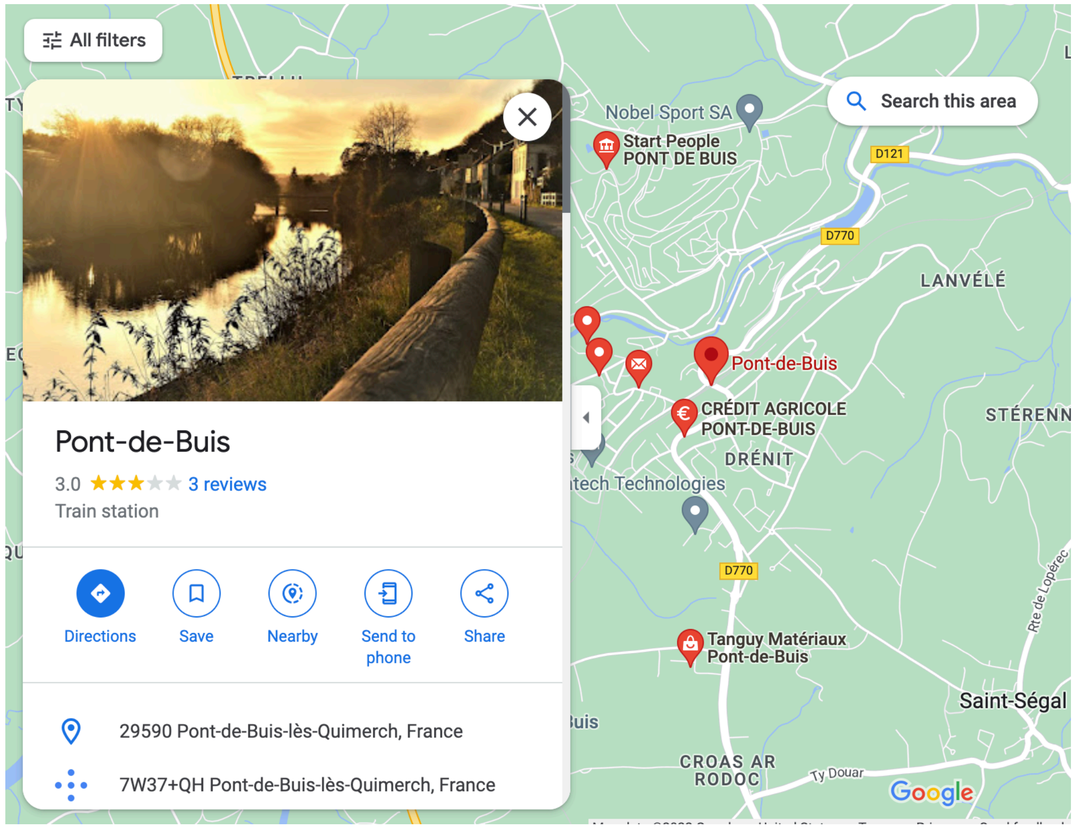
The ecotax portico in Pont-de-Buis, around which a violent demonstration against the tax took place on Saturday, was taken down on Thursday morning.

Rare/Unknown Words

The ecotax portico in Pont-de-Buis, around which a violent demonstration against the tax took place on Saturday, was taken down on Thursday morning.

Rare/Unknown Words

The ecotax portico in Pont-de-Buis, around which a violent demonstration against the tax took place on Saturday, was taken down on Thursday morning.



Limitations of Word-based Models

- Closed vocabulary (100k-300k words is typical)
  - Still, there will be out-of-vocabulary (OOV) words

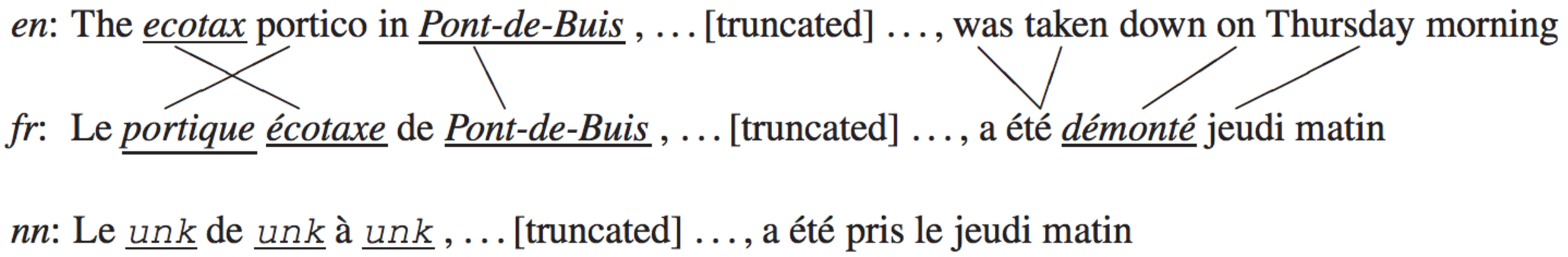
en: The *ecotax* portico in *Pont-de-Buis* , . . . [truncated] . . . , was taken down on Thursday morning

fr: Le *portique* *écotaxe* de *Pont-de-Buis* , . . . [truncated] . . . , a été *démonté* jeudi matin

nn: Le `unk` de `unk` à `unk` , . . . [truncated] . . . , a été pris le jeudi matin

Limitations of Word-based Models

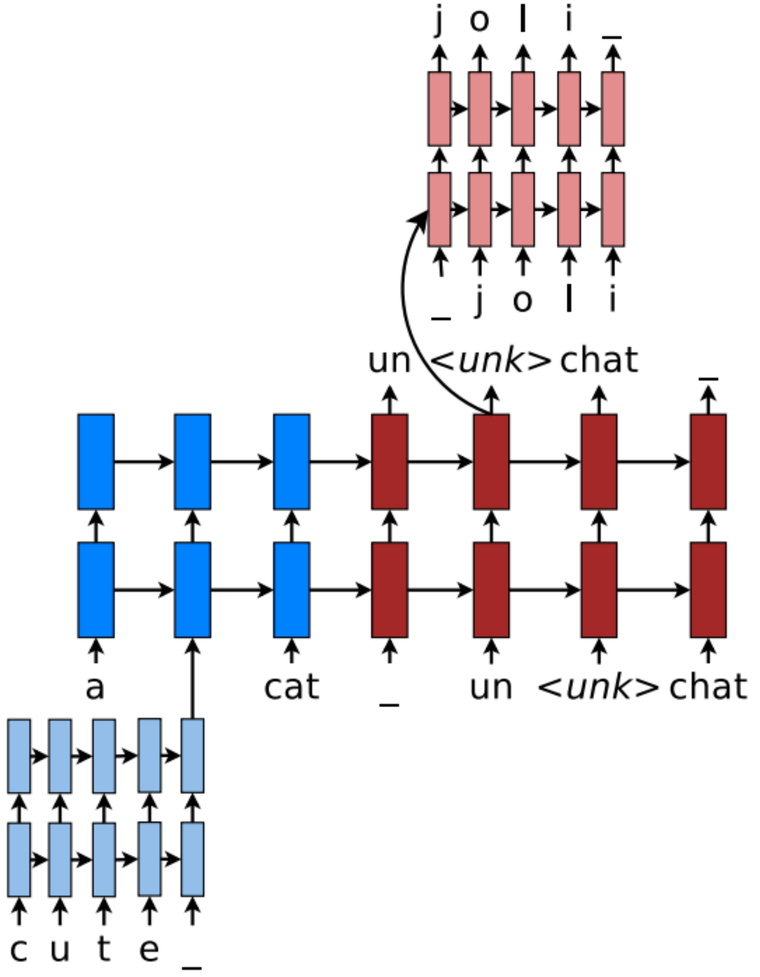
- Closed vocabulary (100k-300k words is typical)
  - Still, there will be out-of-vocabulary (OOV) words



- large number of parameters! (e.g., 100k-300k * 512)
- for morphologically-rich languages, using a separate vector for each word type or character is “obviously” not optimal

Character Models

- If we predict an *unk* token, generate the results from a character LSTM
- Can potentially transliterate new concepts, but architecture is more complicated and slower to train
- Models like this in part contributed to dynamic computation graph frameworks becoming popular



Luong et al. (2016)

Character Models

initialization of parameters in $[-0.1, 0.1]$, (c) 6-epoch training with plain SGD and a simple learning rate schedule – start with a learning rate of 1.0; after 4 epochs, halve the learning rate every 0.5 epoch, (d) mini-batches are of size 128 and shuffled, (e) the gradient is rescaled whenever its norm exceeds 5, and (f) dropout is used with probability 0.2 according to (Pham et al., 2014). We now detail differences across the three architectures.

Word-based NMT – We constrain our source and target sequences to have a maximum length of 50 each; words that go past the boundary are ignored. The vocabularies are limited to the top $|V|$ most frequent words in both languages. Words not in these vocabularies are converted into <*unk*>. After translating, we will perform dictionary[5] lookup or identity copy for <*unk*> using the alignment information from the attention models. Such procedure is referred as the *unk replace* technique (Luong et al., 2015b; Jean et al., 2015a).

Character-based NMT – The source and target sequences at the character level are often about 5 times longer than their counterparts in the word-based models as we can infer from the statistics in Table 1. Due to memory constraint in GPUs, we limit our source and target sequences to a maximum length of 150 each, i.e., we backpropagate through at most 300 timesteps from the decoder to the encoder. With smaller 512-dimensional models, we can afford to have longer sequences with up to 600-step backpropagation.

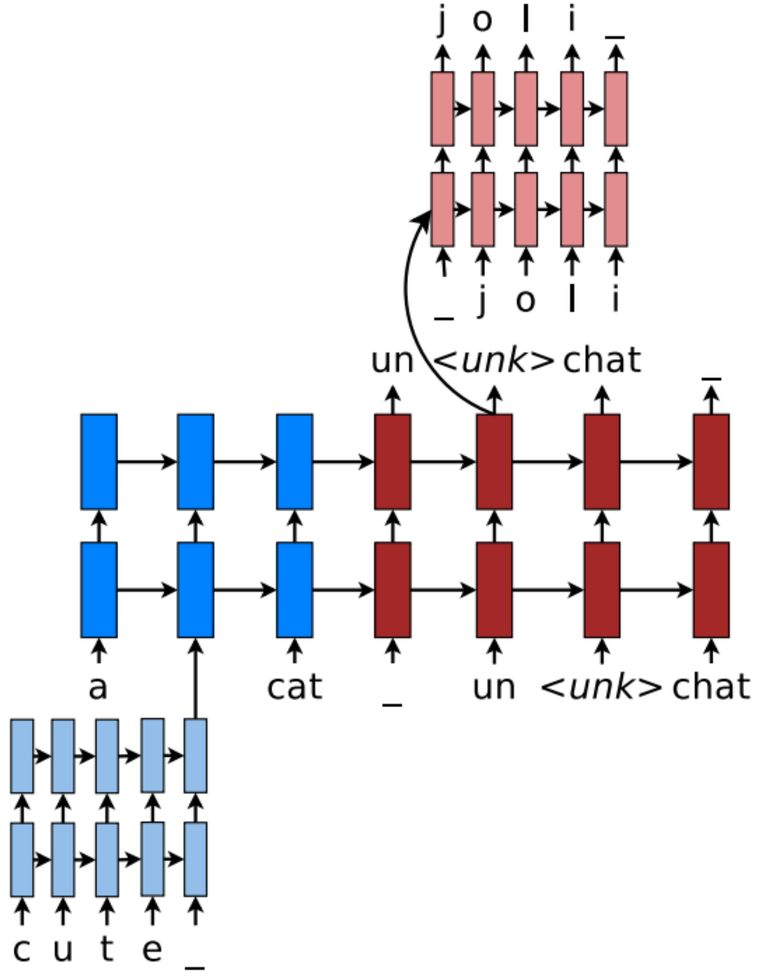
Hybrid NMT – The *word*-level component uses the same settings as the purely word-based NMT. For the *character*-level source and target components, we experiment with both shallow and deep 1024-dimensional models of 1 and 2 LSTM layers. We set the weight α in Eq. (5) for our character-level loss to 1.0.

Training Time – It takes about 3 weeks to train a word-based model with $|V| = 50K$ and about 3 months to train a character-based model. Training and testing for the hybrid models are about 10-20% slower than those of the word-based models with the same vocabulary size.

5.3 Results

We compare our models with several strong systems. These include the winning entry in WMT'15, which was trained on a much larger amount of data, 52.6M parallel and 393.0M mono-

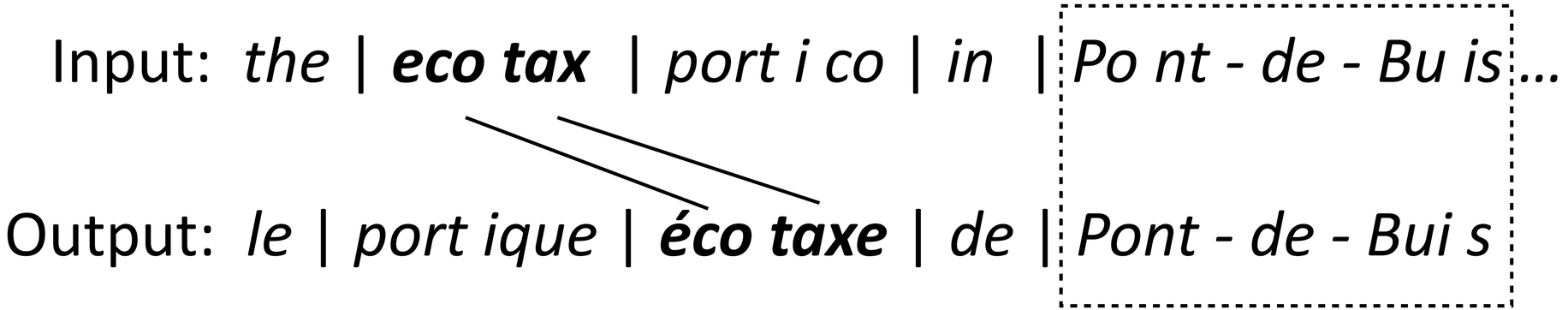
[5]Obtained from the alignment links produced by the Berkeley aligner (Liang et al., 2006) over the training corpus.



Luong et al. (2016)

Handling Rare Words

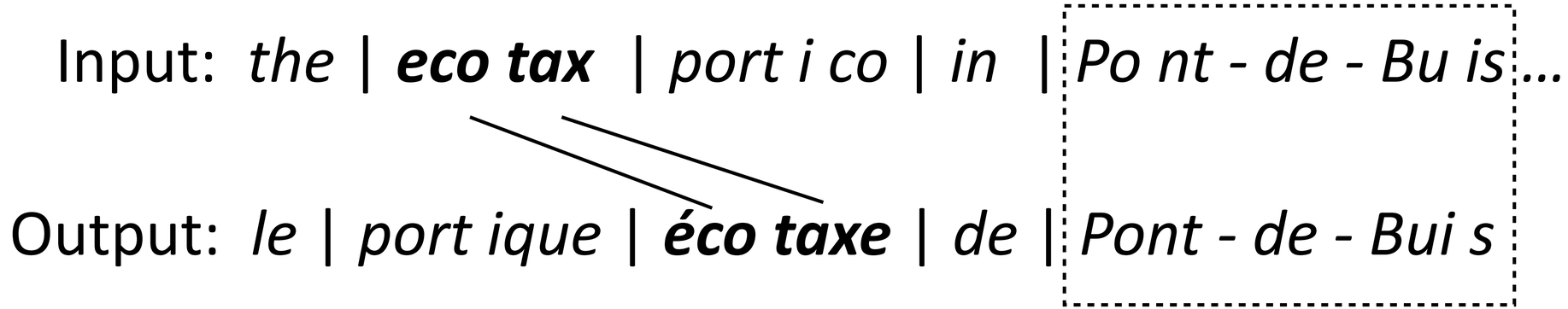
- Words are a difficult unit to work with: copying can be cumbersome, word vocabularies get very large



Wu et al. (2016)

Handling Rare Words

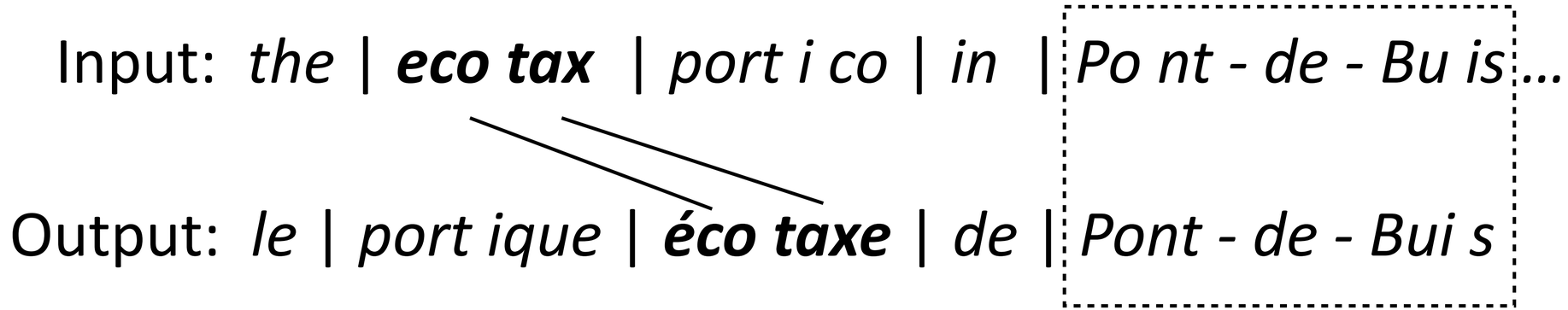
- Words are a difficult unit to work with: copying can be cumbersome, word vocabularies get very large
- Character-level models don't work well



Wu et al. (2016)

Handling Rare Words

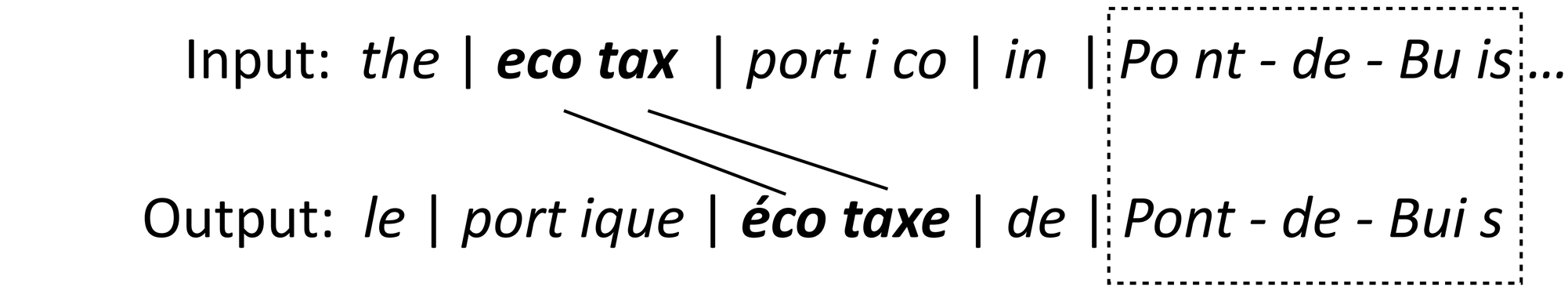
- Words are a difficult unit to work with: copying can be cumbersome, word vocabularies get very large
- Character-level models don't work well
- Solution: "word pieces" (which may be full words but may be subwords)



Wu et al. (2016)

Handling Rare Words

- Words are a difficult unit to work with: copying can be cumbersome, word vocabularies get very large
- Character-level models don't work well
- Solution: "word pieces" (which may be full words but may be subwords)



- Can help with transliteration; capture shared linguistic characteristics between languages (e.g., transliteration, shared word root, etc.)

Wu et al. (2016)

Morphology

What is morphology?

- Study of how words form

What is morphology?

- Study of how words form
- Derivational morphology: create a new *lexeme* from a base

  estrange (v) => estrangement (n)

  become (v) => unbecoming (adj)

  - May not be totally regular: enflame => inflammable

What is morphology?

- Study of how words form
- Derivational morphology: create a new *lexeme* from a base

  estrange (v) => estrangement (n)

  become (v) => unbecoming (adj)

  - May not be totally regular: enflame => inflammable
- Inflectional morphology: word is inflected based on its context

  I become / she become**s**

  - Mostly applies to verbs and nouns

Morphological Inflection

- In English:

I arrive	you arrive	he/she/it arrives	[X] arrived
we arrive	you arrive	they arrive	

Morphological Inflection

- In English: I arrive / you arrive / he/she/it arrives / we arrive / you arrive / they arrive / [X] arrived

- In French:

		singular			plural		
		first	second	third	first	second	third
indicative		je (j')	tu	il, elle	nous	vous	ils, elles
(simple tenses)	present	arrive /a.ʁiv/	arrives /a.ʁiv/	arrive /a.ʁiv/	arrivons /a.ʁi.vɔ̃/	arrivez /a.ʁi.ve/	arrivent /a.ʁiv/
	imperfect	arrivais /a.ʁi.vɛ/	arrivais /a.ʁi.vɛ/	arrivait /a.ʁi.vɛ/	arrivions /a.ʁi.vjɔ̃/	arriviez /a.ʁi.vje/	arrivaient /a.ʁi.vɛ/
	past historic[2]	arrivai /a.ʁi.vɛ/	arrivas /a.ʁi.va/	arriva /a.ʁi.va/	arrivâmes /a.ʁi.vam/	arrivâtes /a.ʁi.vat/	arrivèrent /a.ʁi.vɛʁ/
	future	arriverai /a.ʁi.vʁɛ/	arriveras /a.ʁi.vʁa/	arrivera /a.ʁi.vʁa/	arriverons /a.ʁi.vʁɔ̃/	arriverez /a.ʁi.vʁe/	arriveront /a.ʁi.vʁɔ̃/
	conditional	arriverais /a.ʁi.vʁɛ/	arriverais /a.ʁi.vʁɛ/	arriverait /a.ʁi.vʁɛ/	arriverions /a.ʁi.və.ʁjɔ̃/	arriveriez /a.ʁi.və.ʁje/	arriveraient /a.ʁi.vʁɛ/

Morphological Inflection

- In Spanish:

		singular			plural		
		1st person	2nd person	3rd person	1st person	2nd person	3rd person
indicative		yo	tú vos	él/ella/ello usted	nosotros nosotras	vosotros vosotras	ellos/ellas ustedes
	present	llego	llegas[tú] llegás[vos]	llega	llegamos	llegáis	llegan
	imperfect	llegaba	llegabas	llegaba	llegábamos	llegabais	llegaban
	preterite	llegué	llegaste	llegó	llegamos	llegasteis	llegaron
	future	llegaré	llegarás	llegará	llegaremos	llegaréis	llegarán
	conditional	llegaría	llegarías	llegaría	llegaríamos	llegaríais	llegarían

Noun Inflection

- Not just verbs either; gender, number, case complicate things

Declension of Kind [hide ▲]

	singular			plural	
	indef.	**def.**	**noun**	**def.**	**noun**
nominative	ein	das	Kind	die	Kinder
genitive	eines	des	Kindes, Kinds	der	Kinder
dative	einem	dem	Kind, Kinde[1]	den	Kindern
accusative	ein	das	Kind	die	Kinder

Noun Inflection

- Not just verbs either; gender, number, case complicate things

Declension of Kind [hide ▲]

	singular			plural	
	indef.	**def.**	**noun**	**def.**	**noun**
nominative	ein	das	Kind	die	Kinder
genitive	eines	des	Kindes, Kinds	der	Kinder
dative	einem	dem	Kind, Kinde[1]	den	Kindern
accusative	ein	das	Kind	die	Kinder

- Nominative: I/he/she, accusative: me/him/her, genitive: mine/his/hers

Noun Inflection

- Not just verbs either; gender, number, case complicate things

Declension of Kind [hide ▲]

	singular			plural	
	indef.	**def.**	**noun**	**def.**	**noun**
nominative	ein	das	Kind	die	Kinder
genitive	eines	des	Kindes, Kinds	der	Kinder
dative	einem	dem	Kind, Kinde[1]	den	Kindern
accusative	ein	das	Kind	die	Kinder

- Nominative: I/he/she, accusative: me/him/her, genitive: mine/his/hers
- Dative: merged with accusative in English, shows recipient of something

Noun Inflection

- Not just verbs either; gender, number, case complicate things

Declension of Kind [hide ▲]

	singular			plural	
	indef.	**def.**	**noun**	**def.**	**noun**
nominative	ein	das	Kind	die	Kinder
genitive	eines	des	Kindes, Kinds	der	Kinder
dative	einem	dem	Kind, Kinde[1]	den	Kindern
accusative	ein	das	Kind	die	Kinder

- Nominative: I/he/she, accusative: me/him/her, genitive: mine/his/hers
- Dative: merged with accusative in English, shows recipient of something

  I taught the children <=> Ich unterrichte die Kinder

Noun Inflection

- Not just verbs either; gender, number, case complicate things

Declension of Kind [hide ▲]

	singular			plural	
	indef.	**def.**	**noun**	**def.**	**noun**
nominative	ein	das	Kind	die	Kinder
genitive	eines	des	Kindes, Kinds	der	Kinder
dative	einem	dem	Kind, Kinde[1]	den	Kindern
accusative	ein	das	Kind	die	Kinder

- Nominative: I/he/she, accusative: me/him/her, genitive: mine/his/hers
- Dative: merged with accusative in English, shows recipient of something

  I taught the children <=> Ich unterrichte die Kinder

  I give the children a book <=> Ich gebe ein Buch

Noun Inflection

- Not just verbs either; gender, number, case complicate things

Declension of Kind [hide ▲]

	singular			plural	
	indef.	def.	noun	def.	noun
nominative	ein	das	Kind	die	Kinder
genitive	eines	des	Kindes, Kinds	der	Kinder
dative	einem	dem	Kind, Kinde[1]	den	Kindern
accusative	ein	das	Kind	die	Kinder

- Nominative: I/he/she, accusative: me/him/her, genitive: mine/his/hers
- Dative: merged with accusative in English, shows recipient of something

  I taught the children <=> Ich unterrichte die Kinder

  I give the children a book <=> Ich gebe den Kindern ein Buch

Irregular Inflection

- Common words are often irregular

Irregular Inflection

- Common words are often irregular
  - I am / you are / she is
  - Je suis / tu es / elle est (French)
  - Soy / está / es (Spanish)

Irregular Inflection

- Common words are often irregular
  - I am / you are / she is
  - Je suis / tu es / elle est (French)
  - Soy / está / es (Spanish)
- Less common words typically fall into some regular *paradigm* — these are somewhat predictable

Agglutinating Langauges

- Finnish/Hungarian (Finno-Ugric), also Turkish, Tamil/ Talugu: what a preposition would do in English is instead part of the verb

Turkish	English
Muvaffak	Successful
Muvaffakiyet	Success
Muvaffakiyet**siz**	**Un**successful ('**without** success')
Muvaffakiyetsiz**leş***(-mek)*	*(To)* **become** unsuccessful
Muvaffakiyetsizleş**tir***(-mek)*	*(To)* **make one** unsuccessful
Muvaffakiyetsizleştiri**ci**	**Maker of** unsuccessful ones
Muvaffakiyetsizleştirici**leş***(-mek)*	*(To)* **become** a maker of unsuccessful ones
Muvaffakiyetsizleştiricileş**tir***(-mek)*	*(To)* **make one** a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiri**ver***(-mek)*	*(To)* **easily/quickly** make one a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştirivere**bil***(-mek)*	*(To)* **be able to make** one easily/quickly a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştirivere**meye**bil*(-mek)*	**Not** *(to)* be able to make one easily/quickly a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebil**ecek**	**One who is** not able to make one easily/quickly a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebilecek**ler**	**Those** who are not able to make one easily/quickly a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebilecekleri**miz**	Those who **we** cannot make easily/quickly a maker unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebileceklerimiz**den**	**From** those we can not easily/quickly make a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebileceklerimizden**miş**	**(Would be)** from those we can not easily/quickly make a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebileceklerimizdenmiş**siniz**	**You** would be from those we can not easily/quickly make a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebileceklerimizdenmişsiniz**cesine**	**Like** you would be from those we can not easily/quickly make a maker of unsuccessful ones

Agglutinating Langauges

- Finnish/Hungarian (Finno-Ugric), also Turkish, Tamil/ Talugu: what a preposition would do in English is instead part of the verb

Turkish	English
Muvaffak	Successful
Muvaffakiyet	Success
Muvaffakiyet**siz**	**Un**successful ('**without** success')
Muvaffakiyetsiz**leş***(-mek)*	*(To)* **become** unsuccessful
Muvaffakiyetsizleş**tir***(-mek)*	*(To)* **make one** unsuccessful
Muvaffakiyetsizleştiri**ci**	**Maker of** unsuccessful ones
Muvaffakiyetsizleştirici**leş***(-mek)*	*(To)* **become** a maker of unsuccessful ones
Muvaffakiyetsizleştiricileş**tir***(-mek)*	*(To)* **make one** a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiri**ver***(-mek)*	*(To)* **easily/quickly** make one a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştirivere**bil***(-mek)*	*(To)* **be able to make** one easily/quickly a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştirivere**meye**bil*(-mek)*	**Not** *(to)* be able to make one easily/quickly a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebil**ecek**	**One who is** not able to make one easily/quickly a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebilecek**ler**	**Those** who are not able to make one easily/quickly a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebilecekleri**miz**	Those who **we** cannot make easily/quickly a maker unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebileceklerimiz**den**	**From** those we can not easily/quickly make a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebileceklerimizden**miş**	**(Would be)** from those we can not easily/quickly make a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebileceklerimizdenmiş**siniz**	**You** would be from those we can not easily/quickly make a maker of unsuccessful ones
Muvaffakiyetsizleştiricileştiriveremeyebileceklerimizdenmişsiniz**cesine**	**Like** you would be from those we can not easily/quickly make a maker of unsuccessful ones

- Many possible forms — and in newswire data, only a few are observed

Morphological Analysis: Hungarian

But the government does not recommend reducing taxes.

Ám a kormány egyetlen adó csökkentését sem javasolja .

Morphological Analysis: Hungarian

But the government does not recommend reducing taxes.

Ám a kormány egyetlen adó csökkentését sem javasolja .

n=singular|case=nominative|proper=no

deg=positive|n=singular|case=nominative

n=singular|case=nominative|proper=no

n=singular|case=accusative|proper=no|pperson=3rd|pnumber=singular

mood=indicative|t=present|p=3rd|n=singular|def=yes

4 main types of morphology

- Isolating/Analytical (1 word = 1 morpheme, e.g., Chinese)
- Fusional/Inflectional (1 word = 1 root & some morphemes, e.g., French)
- Agglutinative (1 word = 1 root & many morphemes, e.g., Turkish, Tamil)
- Polysynthetic (1 word = many roots & many morphemes; create full sentence out of morphemes, e.g., Sora)

English	post office
Finnish	postitoimisto
Chinese	邮局

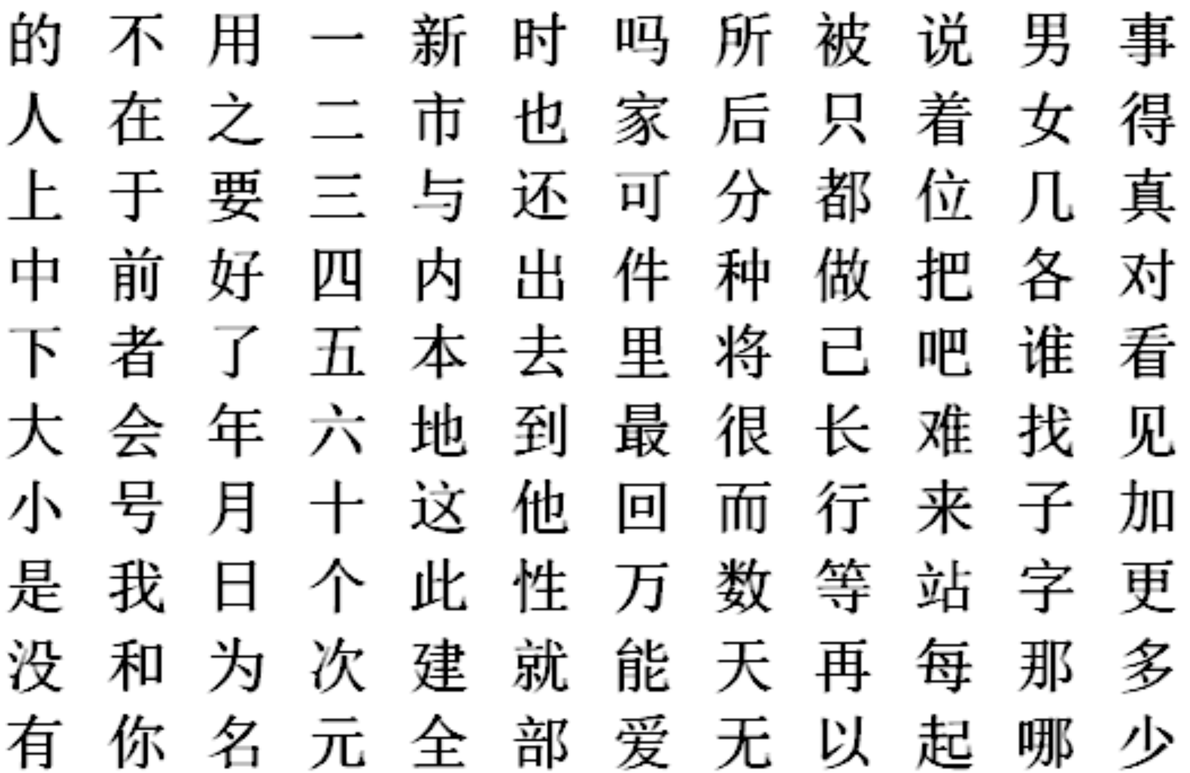
morpheme: the smallest unit of meaning within a word

Morphologically-Rich Languages

- Many languages spoken all over the world have much richer morphology than English
  - CoNLL 2006 / 2007: dependency parsing + morphological analyses for ~15 mostly Indo-European languages
  - SPMRL shared tasks (2013-2014): Syntactic Parsing of Morphologically-Rich Languages
  - Universal Dependencies project (2005-now): >100 languages
- **Word piece / byte-pair encoding** models for MT are pretty good at handling these if there's enough data

Challenges of Chinese

- Thousands of characters!　>80K



Universal Dependencies

- Over 100 languages

Current UD Languages

Information about language families (and genera for families with multiple branches) is mostly taken from WALS Online (IE = Indo-European).

Abaza	1	<1K	Northwest Caucasian
Afrikaans	1	49K	IE, Germanic
Akkadian	2	25K	Afro-Asiatic, Semitic
Akuntsu	1	1K	Tupian, Tupari
Albanian	1	<1K	IE, Albanian
Amharic	1	10K	Afro-Asiatic, Semitic
Ancient Greek	2	416K	IE, Greek
Ancient Hebrew	1	39K	Afro-Asiatic, Semitic
Apurina	1	<1K	Arawakan
Arabic	3	1,042K	Afro-Asiatic, Semitic
Armenian	2	94K	IE, Armenian
Assyrian	1	<1K	Afro-Asiatic, Semitic
Bambara	1	13K	Mande
Basque	1	121K	Basque
Beja	1	<1K	Afro-Asiatic, Cushitic
Belarusian	1	305K	IE, Slavic
Bengali	1	<1K	IE, Indic
Bhojpuri	1	6K	IE, Indic
Breton	1	10K	IE, Celtic
Bulgarian	1	156K	IE, Slavic
Buryat	1	10K	Mongolic
Cantonese	1	13K	Sino-Tibetan
Catalan	1	553K	IE, Romance
Cebuano	1	1K	Austronesian, Central Philippine
Chinese	6	287K	Sino-Tibetan
Chukchi	1	6K	Chukotko-Kamchatkan
Classical Chinese	1	310K	Sino-Tibetan

https://universaldependencies.org/

African Languages!

- AfroLID, a neural LID toolkit for 517 African languages and varieties.

Figure 1: All 50 African countries in our data, with our 517 languages/language varieties in colored circles overlayed within respective countries. More details are in Appendix E.

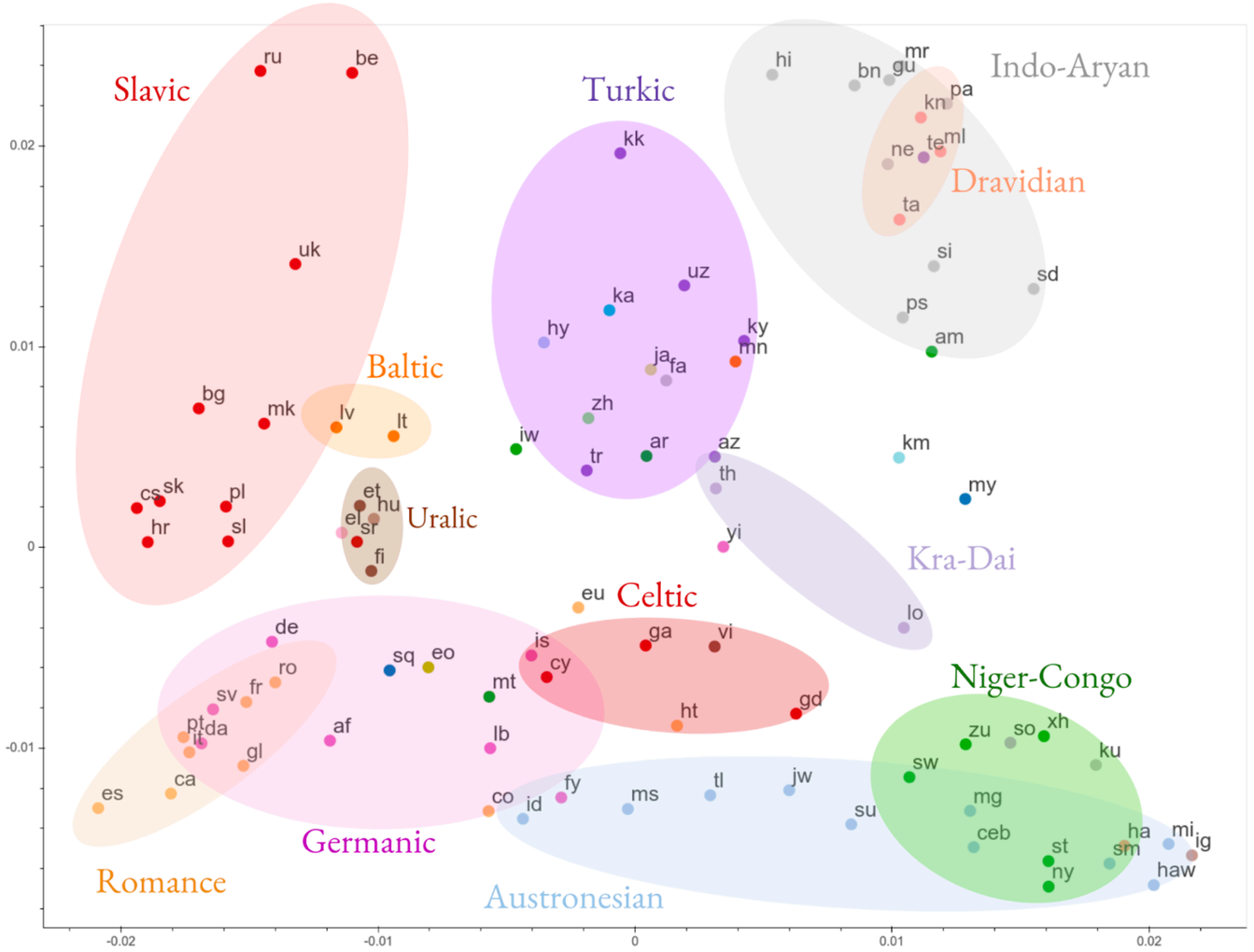
Word Order	Example Languages
SVO	Xhosa, Zulu, Yorùbá
SOV	Khoekhoe, Somali, Amharic
VSO	Murle, Kalenjin
VOS	Malagasy
No-dominant-order	Siswati, Nyamwezi, Bassa

Table 1: Sentential word order in our data.

Adebara et al. (2022)

Massively Multilingual MT

- For 103 languages



Arivazhagan et al. (2019), Kudugunta et al. (2019)

XLM-RoBERTa (XLM-R)

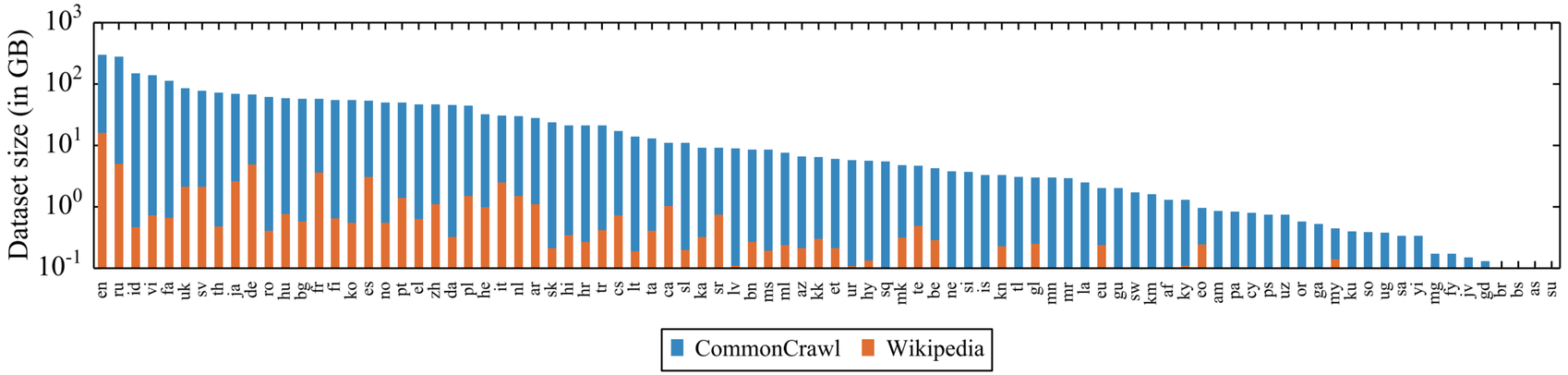


Figure 1: Amount of data in GiB (log-scale) for the 88 languages that appear in both the Wiki-100 corpus used for mBERT and XLM-100, and the CC-100 used for XLM-R. CC-100 increases the amount of data by several orders of magnitude, in particular for low-resource languages.

- Larger “Common Crawl” dataset, better performance than mBERT
- Low-resource languages benefit from training on other languages
- High-resource languages see a small performance hit, but not much

Conneau et al. (2019)

Byte Pair Encoding (BPE)

Byte Pair Encoding (BPE)

- Start with every individual byte (basically character) as its own symbol

Algorithm 1 Byte-pair encoding (Sennrich et al., 2016; Gage, 1994)

```
Input: set of strings D, target vocab size k
procedure BPE(D, k)
    V ← all unique characters in D
          (about 4,000 in English Wikipedia)
    while |V| < k do                ▷ Merge tokens
        t_L, t_R ← Most frequent bigram in D
        t_NEW ← t_L + t_R           ▷ Make new token
        V ← V + [t_NEW]
        Replace each occurrence of t_L, t_R in
            D with t_NEW
    end while
    return V
end procedure
```

Sennrich et al. (2016); Figure from Bostrom and Durrett (2020)

Byte Pair Encoding (BPE)

- Start with every individual byte (basically character) as its own symbol
- Count bigram character cooccurrences

Algorithm 1 Byte-pair encoding (Sennrich et al., 2016; Gage, 1994)

```
Input: set of strings D, target vocab size k
procedure BPE(D, k)
    V ← all unique characters in D
          (about 4,000 in English Wikipedia)
    while |V| < k do                ▷ Merge tokens
        t_L, t_R ← Most frequent bigram in D
        t_NEW ← t_L + t_R           ▷ Make new token
        V ← V + [t_NEW]
        Replace each occurrence of t_L, t_R in
            D with t_NEW
    end while
    return V
end procedure
```

Sennrich et al. (2016); Figure from Bostrom and Durrett (2020)

Byte Pair Encoding (BPE)

- Start with every individual byte (basically character) as its own symbol
- Count bigram character cooccurrences
- Merge the most frequent pair of adjacent characters

Algorithm 1 Byte-pair encoding (Sennrich et al., 2016; Gage, 1994)

```
Input: set of strings D, target vocab size k
procedure BPE(D, k)
    V ← all unique characters in D
          (about 4,000 in English Wikipedia)
    while |V| < k do                ▷ Merge tokens
        t_L, t_R ← Most frequent bigram in D
        t_NEW ← t_L + t_R           ▷ Make new token
        V ← V + [t_NEW]
        Replace each occurrence of t_L, t_R in
            D with t_NEW
    end while
    return V
end procedure
```

Sennrich et al. (2016); Figure from Bostrom and Durrett (2020)

Training & Inference

Training:

- Step1. Pre-tokenization
  - Assumptions and heuristics before encoding (split on white spaces)
- Step2. Vocabulary Initialization
  - Initialize with all unique characters after pre-tokenization
- Step3. Learning Merge Rules
  - Form sub-words to maximize compression (frequency-based objective)

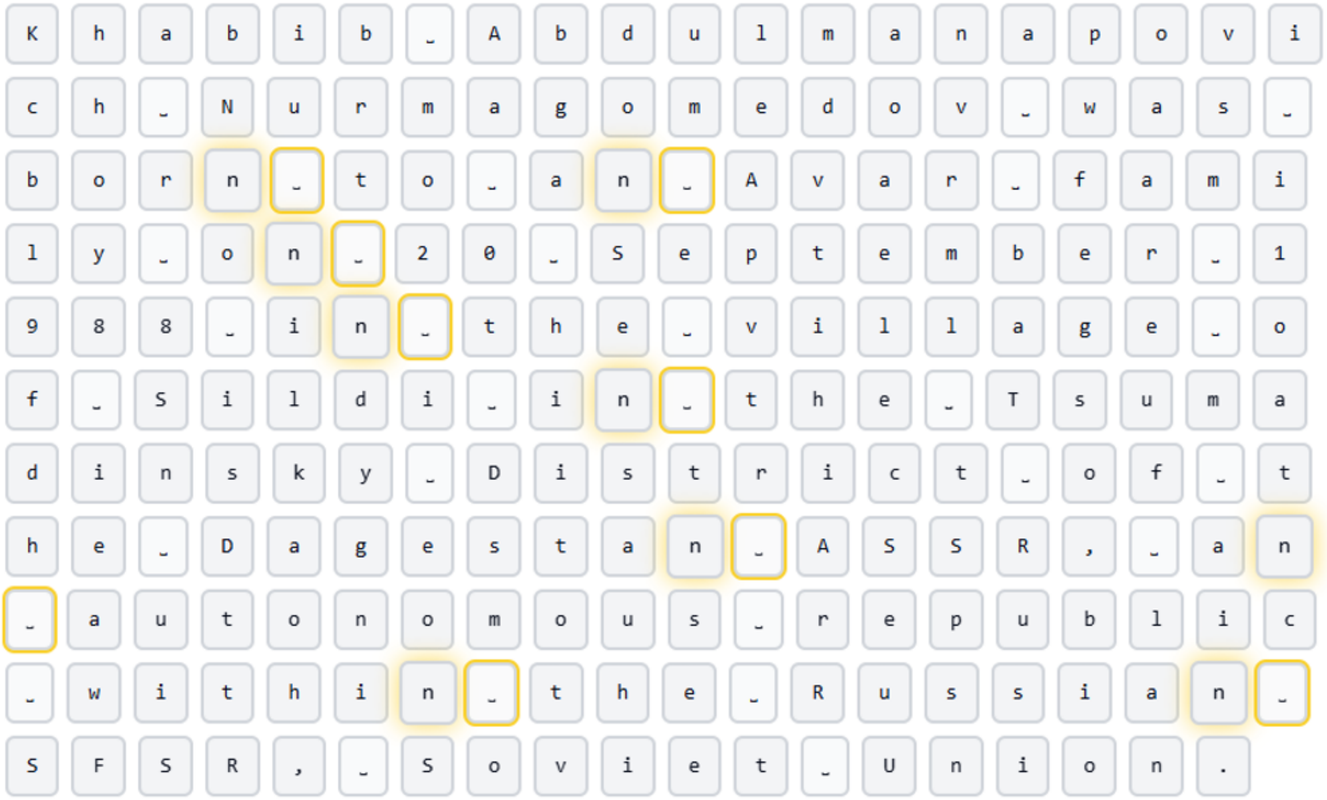
Inference:

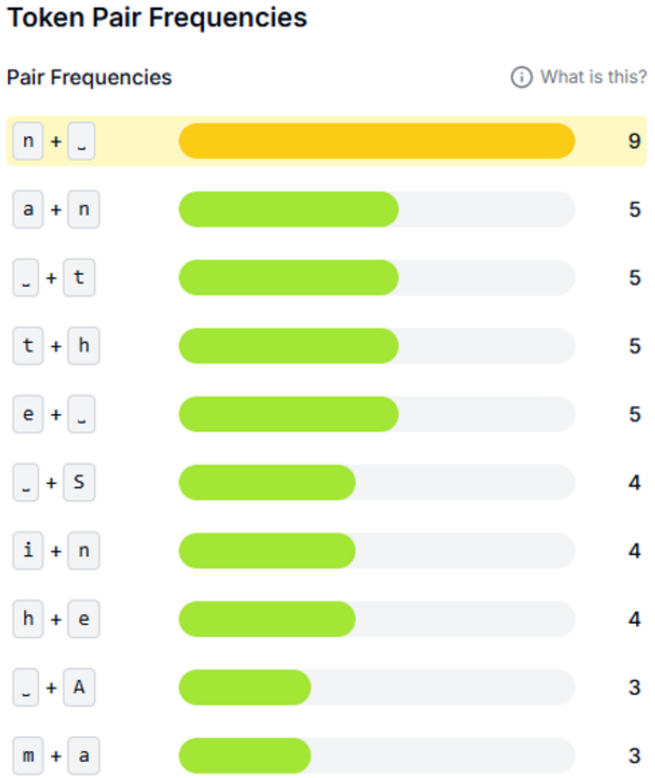
- Inference algorithm: decide how to tokenize by applying the merge rules

Slide Credit: Tarek Naous

An Example

Merge Rules: Step 1 – Count pair frequencies





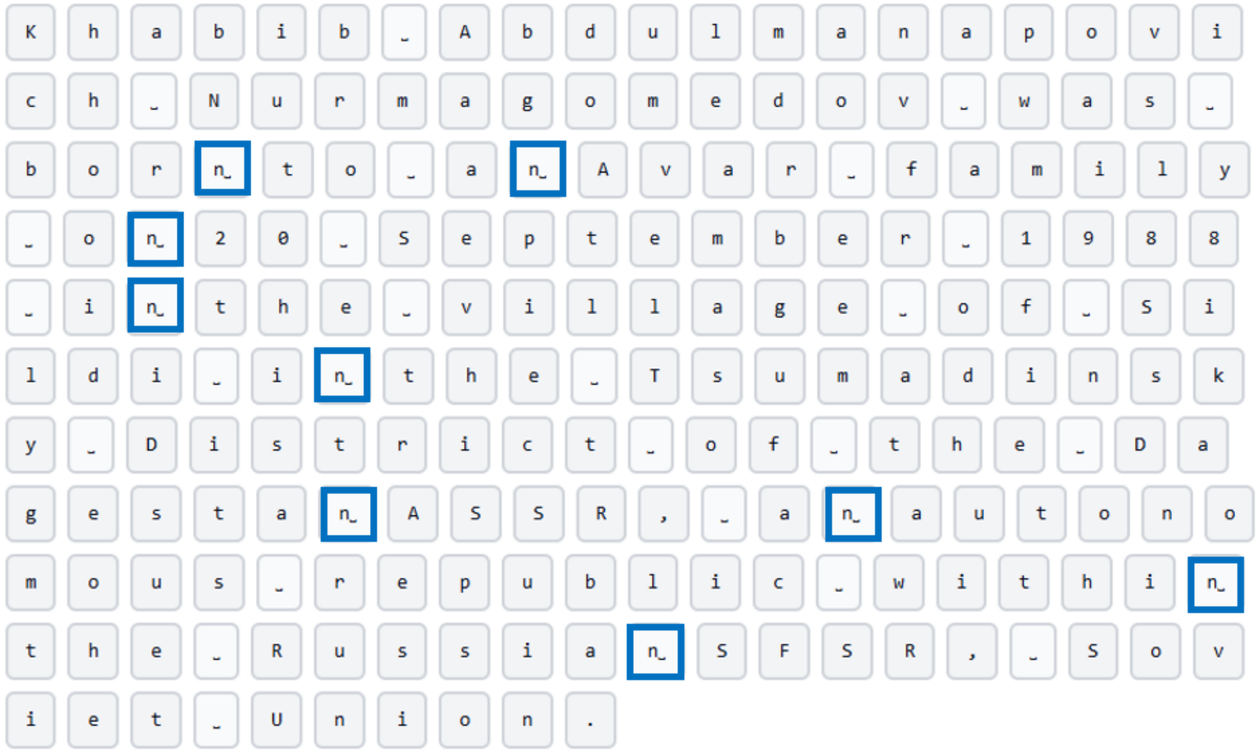
The most frequent pair is merged into a new token: n + _ → n_

Note that, in current tokenizers, we do not cross the space boundaries when forming merges

(Though in this visualization tool it does cross boundaries, as you can see from some pair counts)

An Example

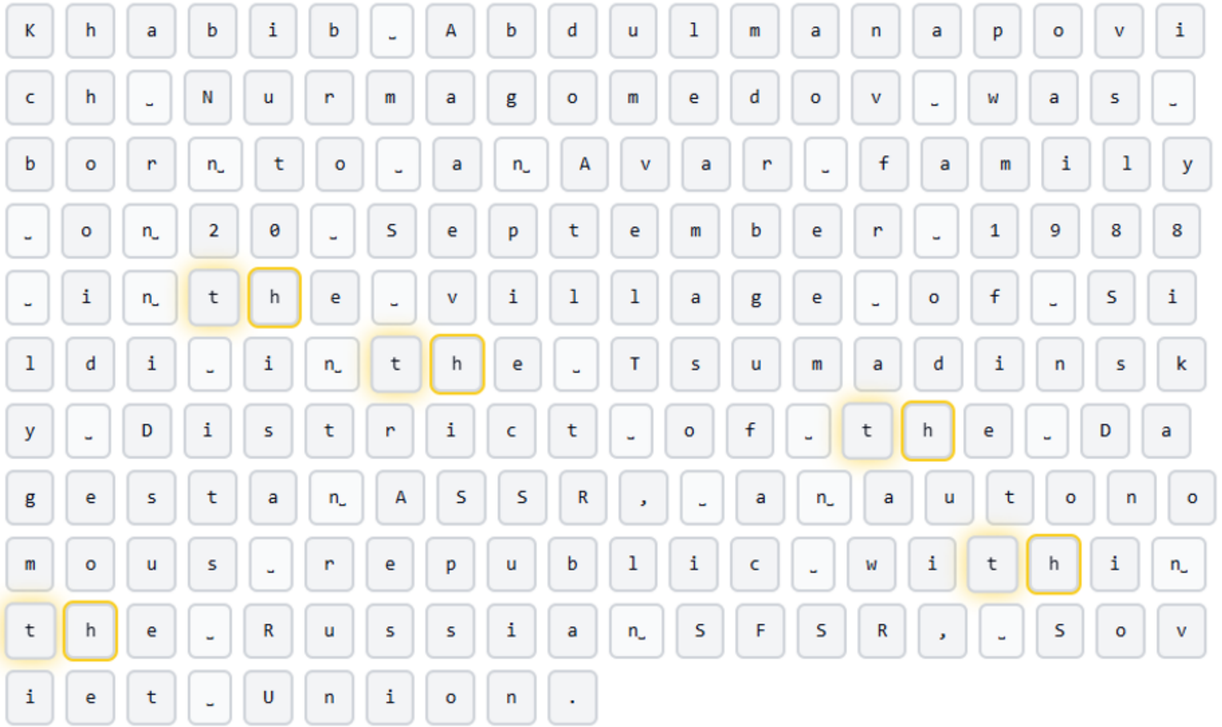
Merge Rules: Step 2 – Apply merge to corpus

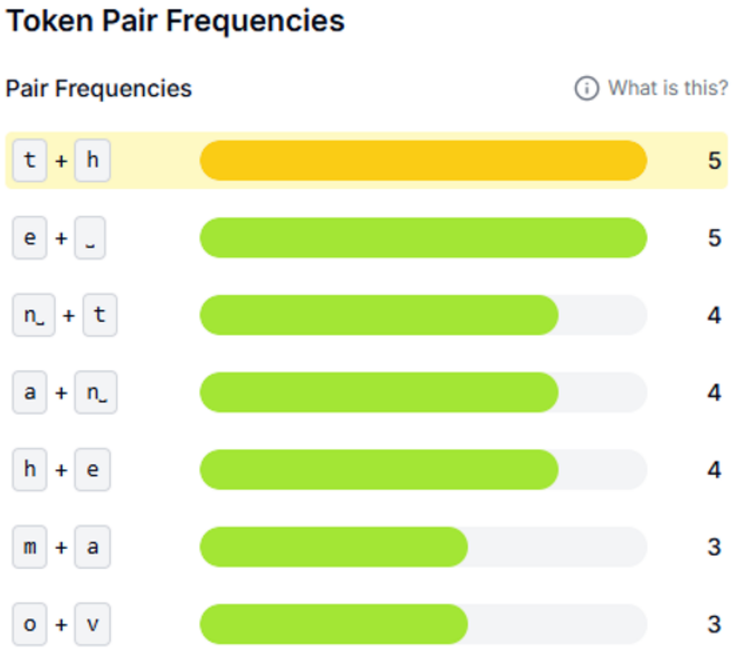


The past merge is first applied to the corpus: n + _ → n_

An Example

Merge Rules: Step 3 – Count pair frequencies again





Recompute pair counts on corpus to identify new most frequent pair.

Add the identified pair to the vocabulary as a merge rule.

This is repeatedly done until we reach a desired vocabulary size K.

An Example

Inference Algorithm: Apply Merge Rules to Tokenize

Merge Rules

a
b
c
..
y
z
…
b_
…
a b → ab
m e → me
…
h ab → hab
me d → med
…

An Example

Inference Algorithm: Apply Merge Rules to Tokenize

Khabib Nurmagomedov

Merge Rules

a
b
c
..
y
z
…
b_
…
a b → ab
m e → me
…
h ab → hab
me d → med
…

An Example

Inference Algorithm: Apply Merge Rules to Tokenize

Merge Rules

```
a
b
c
..
y
z
…
b_
…
a b → ab
m e → me
…
h ab → hab
me d → med
…
```

Khabib Nurmagomedov

Khabib_ Nurmagomedov

First split on whitepace and see if word is already a token in vocab

An Example

Inference Algorithm: Apply Merge Rules to Tokenize

Merge Rules

a
b
c
..
y
z
…
b_
…
a b → ab
m e → me
…
h ab → hab
me d → med
…

Khabib Nurmagomedov

Khabib_ Nurmagomedov

First split on whitepace and see if word is already a token in vocab

K h a b i b_ N u r m a g o m e d o v

Split by character if not

An Example

Inference Algorithm: Apply Merge Rules to Tokenize

Merge Rules

a
b
c
..
y
z
…
b_
…
a b → ab
m e → me
…
h ab → hab
me d → med
…

Khabib Nurmagomedov

Khabib_ Nurmagomedov

First split on whitepace and see if word is already a token in vocab

K h a b i b_ N u r m a g o m e d o v

Split by character if not

K h ab i b_ N u r m a g o me d o v

Apply merge rules in order

An Example

Inference Algorithm: Apply Merge Rules to Tokenize

Merge Rules

a
b
c
..
y
z
…
b_
…
a b → ab
m e → me
…
h ab → hab
me d → med
…

Khabib Nurmagomedov

Khabib_ Nurmagomedov

First split on whitepace and see if word is already a token in vocab

K h a b i b_ N u r m a g o m e d o v

Split by character if not

K h ab i b_ N u r m a g o me d o v

Apply merge rules in order

K hab i b_ N u r m a g o med o v

An Example

Inference Algorithm: Apply Merge Rules to Tokenize

Merge Rules

a
b
c
..
y
z
…
b_
…
a b → ab
m e → me
…
h ab → hab
me d → med
…

Khabib Nurmagomedov

Khabib_ Nurmagomedov

First split on whitepace and see if word is already a token in vocab

K h a b i b_ N u r m a g o m e d o v

Split by character if not

K h ab i b_ N u r m a g o me d o v

Apply merge rules in order

K hab i b_ N u r m a g o med o v

Tokenized text: *(14 tokens in this example)*

['K', 'hab', 'i', 'b_', 'N', 'u', 'r', 'm', 'a', 'g', 'o', 'med', 'o', 'v']

An Example

Inference Algorithm: Apply Merge Rules to Tokenize

Merge Rules

a
b
c
..
y
z
…
b_
…
a b → ab
m e → me
…
h ab → hab
me d → med
…

Khabib Nurmagomedov

Khabib_ Nurmagomedov

First split on whitepace and see if word is already a token in vocab

K h a b i b_ N u r m a g o m e d o v

Split by character if not

K h ab i b_ N u r m a g o me d o v

Apply merge rules in order

K hab i b_ N u r m a g o med o v

Tokenized text: *(14 tokens in this example)*

['K', 'hab', 'i', 'b_', 'N', 'u', 'r', 'm', 'a', 'g', 'o', 'med', 'o', 'v']

Compression rate = $\frac{18\ (chars)}{14\ (tokens)} = 1.285$

Byte, not Characters

- But, more accurately —

  Byte Pair Encoding (BPE) applies to **byte**, not characters

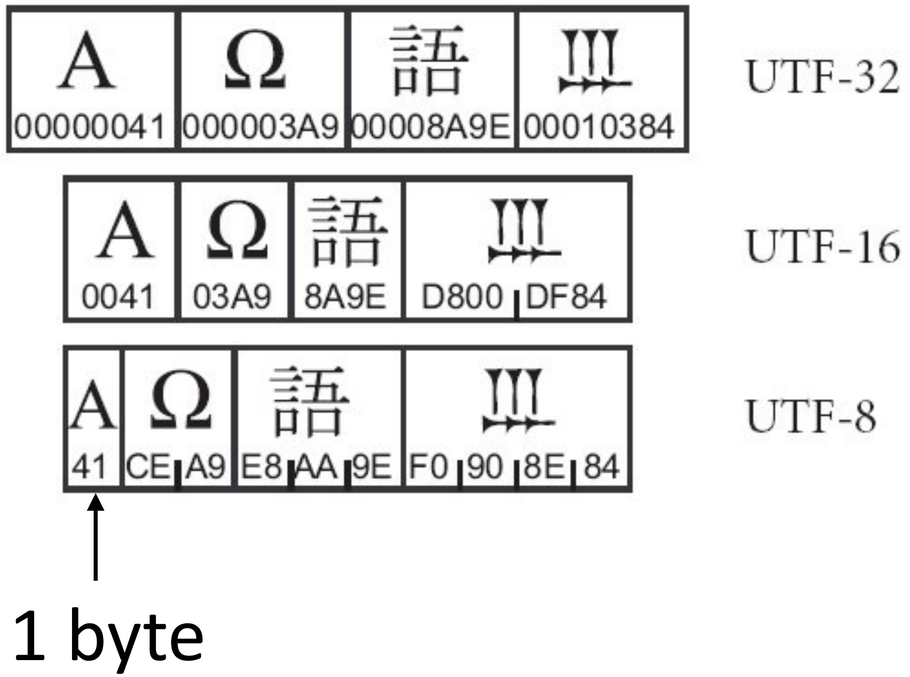
- What is the problem with directly using text strings?

Byte, not Characters

- What is the problem with directly using text strings?
  - Large vocabulary: considering all writing scripts, the initialization step by itself will make the vocabulary tens of thousands long
  - Unseen/rare characters: <unk> problem again
  - Other issues: string encoding mismatch at test time, worse multilingual support, etc.

Unicode —> Byte Representation

- UTF-8: variable-length encoding (1-4 bytes)
- UTF-16: fixed- and variable-length hybrid (2-4 bytes)
- UTF-32: fixed-length encoding (everything is 4 bytes)



Unicode

- Unicode is a text encoding standard that maps each character to an integer, which is called “code point”
- Version 17.0 (Sep 2025) defines **159,801** unique characters and 172 scripts.

```
1 print(ord('a'), ord('त'), ord('智'), ord('🤗'))
```

```
97 2340 26234 129303
```

Unicode

- Version 17.0 (Sep 2025) defines 159,801 unique characters and 172 scripts.

Oct	Binary	Char	Name
0	0000 0000	NUL	Null
1	0000 0001	SOH	Start of heading
2	0000 0010	STX	Start of text
3	0000 0011	ETX	End of text
4	0000 0100	EOT	End of transmission
5	0000 0101	ENQ	Enquiry
6	0000 0110	ACK	Acknowledge
7	0000 0111	BEL	Bell
8	0000 1000	BS	Backspace
9	0000 1001	HT	Horizontal tab
10	0000 1010	LF	NL line feed, new line
11	0000 1011	VT	Vertical tab
12	0000 1100	FF	NP form feed, new page
13	0000 1101	CR	Carriage return
14	0000 1110	SO	Shift out
15	0000 1111	SI	Shift in
16	0001 0000	DLE	Data link escape
17	0001 0001	DC1	Device control 1
18	0001 0010	DC2	Device control 2
19	0001 0011	DC3	Device control 3
20	0001 0100	DC4	Device control 4
21	0001 0101	NAK	Negative acknowledge
22	0001 0110	SYN	Synchronous idle
23	0001 0111	ETB	End of trans. block
24	0001 1000	CAN	Cancel
25	0001 1001	EM	End of medium
26	0001 1010	SUB	Substitute
27	0001 1011	ESC	Escape
28	0001 1100	FS	File separator
29	0001 1101	GS	Group separator
30	0001 1110	RS	Record separator
31	0001 1111	US	Unit separator
32	0010 0000	SP	Space

Oct	Binary	Char
33	0010 0001	!
34	0010 0010	"
35	0010 0011	#
36	0010 0100	$
37	0010 0101	%
38	0010 0110	&
39	0010 0111	'
40	0010 1000	(
41	0010 1001	)
42	0010 1010	*
43	0010 1011	+
44	0010 1100	,
45	0010 1101	-
46	0010 1110	.
47	0010 1111	/
48	0011 0000	0
49	0011 0001	1
50	0011 0010	2
51	0011 0011	3
52	0011 0100	4
53	0011 0101	5
54	0011 0110	6
55	0011 0111	7
56	0011 1000	8
57	0011 1001	9
58	0011 1010	:
59	0011 1011	;
60	0011 1100	<
61	0011 1101	=
62	0011 1110	>
63	0011 1111	?
64	0100 0000	@

Oct	Binary	Char
65	0100 0001	A
66	0100 0010	B
67	0100 0011	C
68	0100 0100	D
69	0100 0101	E
70	0100 0110	F
71	0100 0111	G
72	0100 1000	H
73	0100 1001	I
74	0100 1010	J
75	0100 1011	K
76	0100 1100	L
77	0100 1101	M
78	0100 1110	N
79	0100 1111	O
80	0101 0000	P
81	0101 0001	Q
82	0101 0010	R
83	0101 0011	S
84	0101 0100	T
85	0101 0101	U
86	0101 0110	V
87	0101 0111	W
88	0101 1000	X
89	0101 1001	Y
90	0101 1010	Z
91	0101 1011	[
92	0101 1100	\
93	0101 1101	]
94	0101 1110	^
95	0101 1111	_
96	0110 0000	`

Oct	Binary	Char
97	0110 0001	a
98	0110 0010	b
99	0110 0011	c
100	0110 0100	d
101	0110 0101	e
102	0110 0110	f
103	0110 0111	g
104	0110 1000	h
105	0110 1001	i
106	0110 1010	j
107	0110 1011	k
108	0110 1100	l
109	0110 1101	m
110	0110 1110	n
111	0110 1111	o
112	0111 0000	p
113	0111 0001	q
114	0111 0010	r
115	0111 0011	s
116	0111 0100	t
117	0111 0101	u
118	0111 0110	v
119	0111 0111	w
120	0111 1000	x
121	0111 1001	y
122	0111 1010	z
123	0111 1011	{
124	0111 1100	|
125	0111 1101	}
126	0111 1110	~
127	0111 1111	DEL

www.hello-algo.com

1 byte = 8 bits

UTF-8

- Most commonly used, back-compatible with ASCII
- Variable length encoding that uses 1 to 4 bytes per character depending on where it falls in the range

Code point ↔ UTF-8 conversion

First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+007F	0yyyzzzz			
U+0080	U+07FF	110xxxyy	10yyzzzz		
U+0800	U+FFFF	1110wwww	10xxxxyy	10yyzzzz	
U+010000	U+10FFFF	11110uvv	10vvwwww	10xxxxyy	10yyzzzz

UTF-8

- Most commonly used, back-compatible with ASCII
- Variable length encoding that uses 1 to 4 bytes per character depending on where it falls in the range

Code point ↔ UTF-8 conversion

First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+007F	0yyyzzzz			
U+0080	U+07FF	110xxxyy	10yyzzzz		
U+0800	U+FFFF	1110wwww	10xxxxyy	10yyzzzz	
U+010000	U+10FFFF	11110uvv	10vvwwww	10xxxxyy	10yyzzzz

UTF-8

- Most commonly used, back-compatible with ASCII
- Variable length encoding that uses 1 to 4 bytes per character depending on where it falls in the range

Code point ↔ UTF-8 conversion

First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+007F	0yyyzzzz			
U+0080	U+07FF	110xxxyy	10yyzzzz		
U+0800	U+FFFF	1110wwww	10xxxxyy	10yyzzzz	
U+010000	U+10FFFF	11110uvv	10vvwwww	10xxxxyy	10yyzzzz

UTF-8

- Most commonly used, back-compatible with ASCII
- Variable length encoding that uses 1 to 4 bytes per character depending on where it falls in the range

Code point ↔ UTF-8 conversion

First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4	# free bits
U+0000	U+007F	0yyyzzzz				7
U+0080	U+07FF	110xxxyy	10yyzzzz			11
U+0800	U+FFFF	1110wwww	10xxxxyy	10yyzzzz		16
U+010000	U+10FFFF	11110uvv	10vvwwww	10xxxxyy	10yyzzzz	21

UTF-8

- Most commonly used, back-compatible with ASCII
- Variable length encoding that uses 1 to 4 bytes per character depending on where it falls in the range

Code point ↔ UTF-8 conversion

First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+007F	0yyyzzzz			
U+0080	U+07FF	110xxxyy	10yyzzzz		
U+0800	U+FFFF	1110wwww	10xxxxyy	10yyzzzz	
U+010000	U+10FFFF	11110uvv	10vvwwww	10xxxxyy	10yyzzzz

# free bits	max
7	007F (127)
11	07FF (2,047)
16	FFFF (65,535)
21	10FFFF (1,114,111)

UTF-8

- Using this encoding scheme, we can represent any character as a list of bytes:

Character	Codepoint (Hex)	Codepoint (Decimal)	Bytes
T	U+0054	84	[84]
ش	U+0634	1588	[216, 180]
𡨸	U+21A38	137784	[240, 161, 168, 184]

Byte, not Characters

- Why use raw bytes as base units instead of characters?

One Example

Khabib Abdulmanapovich Nurmagomedov was born to an Avar family on 20 September 1988 in the village of Sildi in the Tsumadinsky District of the Dagestan ASSR, an autonomous republic within the Russian SFSR, Soviet Union.

```
[75, 104, 97, 98, 105, 98, 32, 65, 98, 100, 117, 108, 109, 97, 110, 97, 112, 111, 118, 105, 99, 104, 32, 78, 117, 114,
109, 97, 103, 111, 109, 101, 100, 111, 118, 32, 119, 97, 115, 32, 98, 111, 114, 110, 32, 116, 111, 32, 97, 110, 32,
65, 118, 97, 114, 32, 102, 97, 109, 105, 108, 121, 32, 111, 110, 32, 50, 48, 32, 83, 101, 112, 116, 101, 109, 98, 101,
114, 32, 49, 57, 56, 56, 32, 105, 110, 32, 116, 104, 101, 32, 118, 105, 108, 108, 97, 103, 101, 32, 111, 102, 32, 83,
105, 108, 100, 105, 32, 105, 110, 32, 116, 104, 101, 32, 84, 115, 117, 109, 97, 100, 105, 110, 115, 107, 121, 32, 68,
105, 115, 116, 114, 105, 99, 116, 32, 111, 102, 32, 116, 104, 101, 32, 68, 97, 103, 101, 115, 116, 97, 110, 32, 65,
83, 83, 82, 44, 32, 97, 110, 32, 97, 117, 116, 111, 110, 111, 109, 111, 117, 115, 32, 114, 101, 112, 117, 98, 108,
105, 99, 32, 119, 105, 116, 104, 105, 110, 32, 116, 104, 101, 32, 82, 117, 115, 115, 105, 97, 110, 32, 83, 70, 83, 82,
44, 32, 83, 111, 118, 105, 101, 116, 32, 85, 110, 105, 111, 110, 46]
```

Text will be represented as bytes, then the same BPE process is applied

In this case, both the string and byte representation have the same length (219), since 1 latin character corresponds to 1 byte in UTF-8

Another Example

تتميز الثورة السورية بأنها أوّل ثورة مكتملة الأركان منذ الربيع العربي، وأنها ثورة قادرة على إحداث تغييرات جذرية في بنية النظام، بما يوفر فرصًا حقيقية لحالة صعود نهضوي كبير

```
[216, 170, 216, 170, 217, 133, 217, 138, 216, 178, 32, 216, 167, 217, 132, 216, 171, 217, 136, 216, 177, 216, 169, 32,
216, 167, 217, 132, 216, 179, 217, 136, 216, 177, 217, 138, 216, 169, 32, 216, 168, 216, 163, 217, 134, 217, 135, 216,
167, 32, 216, 163, 217, 136, 217, 145, 217, 142, 217, 132, 32, 216, 171, 217, 136, 216, 177, 216, 169, 32, 217, 133,
217, 131, 216, 170, 217, 133, 217, 132, 216, 169, 32, 216, 167, 217, 132, 216, 163, 216, 177, 217, 131, 216, 167, 217,
134, 32, 217, 133, 217, 134, 216, 176, 32, 216, 167, 217, 132, 216, 177, 216, 168, 217, 138, 216, 185, 32, 216, 167,
217, 132, 216, 185, 216, 177, 216, 168, 217, 138, 216, 140, 32, 217, 136, 216, 163, 217, 134, 217, 135, 216, 167, 32,
216, 171, 217, 136, 216, 177, 216, 169, 32, 217, 130, 216, 167, 216, 175, 216, 177, 216, 169, 32, 216, 185, 217, 132,
217, 137, 32, 216, 165, 216, 173, 216, 175, 216, 167, 216, 171, 32, 216, 170, 216, 186, 217, 138, 217, 138, 216, 177,
216, 167, 216, 170, 32, 216, 172, 216, 176, 216, 177, 217, 138, 216, 169, 32, 217, 129, 217, 138, 32, 216, 168, 217,
134, 217, 138, 216, 169, 32, 216, 167, 217, 132, 217, 134, 216, 184, 216, 167, 217, 133, 216, 140, 32, 216, 168, 217,
133, 216, 167, 32, 217, 138, 217, 136, 217, 129, 216, 177, 32, 217, 129, 216, 177, 216, 181, 217, 139, 216, 167, 32,
216, 173, 217, 130, 217, 138, 217, 130, 217, 138, 216, 169, 32, 217, 132, 216, 173, 216, 167, 217, 132, 216, 169, 32,
216, 181, 216, 185, 217, 136, 216, 175, 32, 217, 134, 217, 135, 216, 182, 217, 136, 217, 138, 32, 217, 131, 216, 168,
217, 138, 216, 177, 46]
```

In this Arabic text example, 1 character typically corresponds to 2 bytes

String representation length: 173 characters

Byte representation length: 317 bytes

Non-English and specifically non-latin script languages will typically be fragmented into more tokens, just because their order in the UTF-8 code point range is higher

Byte Pair Encoding (BPE)

- Do this either over your vocabulary (original version) or over a large corpus (more common version)
- Final vocabulary size is often in 10k ~ 30k range for each language
- BPE tokenization takes the vocabulary V containing ordered merges and applies them to new text in the same order as they occurred during vocabulary construction

Sennrich et al. (2016)

Word Pieces

while voc size < target voc size:

Build a language model over your corpus

Merge pieces that lead to highest improvement in language model perplexity

Sennrich et al. (2016), Kudo (2018)

Word Pieces

while voc size < target voc size:

Build a language model over your corpus

Merge pieces that lead to highest improvement in language model perplexity

- SentencePiece library from Google: unigram LM & BPE

Sennrich et al. (2016), Kudo (2018)

Word Pieces

while voc size < target voc size:

Build a language model over your corpus

Merge pieces that lead to highest improvement in language model perplexity

- Result: way of segmenting input appropriate for translation
- SentencePiece library from Google: unigram LM & BPE

Sennrich et al. (2016), Kudo (2018)

Word Pieces

- Alternatively, can learn word pieces based on unigram LM:

  while voc size < target voc size:

  Build a language model over your corpus

  Merge pieces that lead to highest improvement in language model perplexity

- Result: way of segmenting input appropriate for translation
- SentencePiece library from Google: unigram LM & BPE

Sennrich et al. (2016), Kudo (2018)

Word Pieces

- Alternatively, can learn word pieces based on unigram LM:

  while voc size < target voc size:

  Build a language model over your corpus

  Merge pieces that lead to highest improvement in language model perplexity

- Result: way of segmenting input appropriate for translation
- SentencePiece library from Google: unigram LM & BPE
- Large pre-trained language models are all using this now!

Sennrich et al. (2016), Kudo (2018)

Comparison

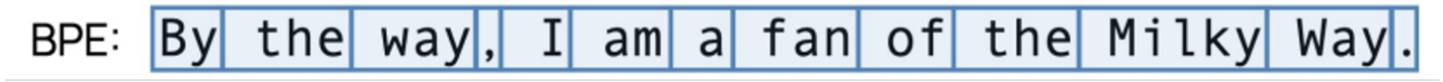
(a)	**Original:**	furiously
	BPE:	_fur | ious | ly
	Unigram LM:	_fur | ious | ly
(b)	**Original:**	tricycles
	BPE:	_t | ric | y | cles
	Unigram LM:	_tri | cycle | s
(c)	**Original:**	Completely preposterous suggestions
	BPE:	_Comple | t | ely | _prep | ost | erous | _suggest | ions
	Unigram LM:	_Complete | ly | _pre | post | er | ous | _suggestion | s

- BPE produces less linguistically plausible units than word pieces (based on unigram LM)
- Some evidence that unigram LM works better in pre-trained Transformer models

Bostrom and Durrett (2020)

SuperBPE

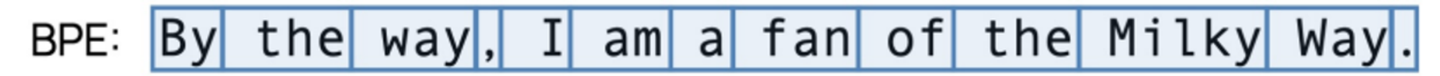
- BPE tokenization is done at the level of subwords, meaning that tokens consist of parts of words (including complete words)
- Although seemingly reasonable, is this common practice a good one?

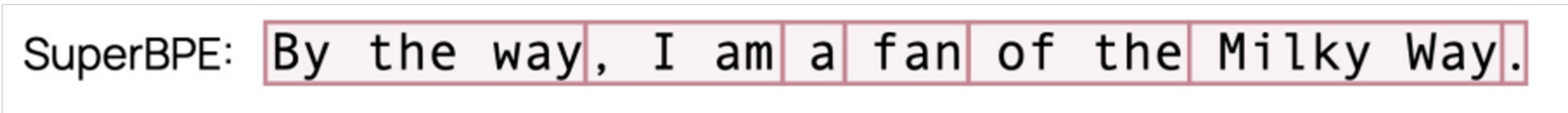


Liu et al. (2025)

SuperBPE

- BPE tokenization is done at the level of subwords, meaning that tokens consist of parts of words (including complete words)
- Although seemingly reasonable, is this common practice a good one?



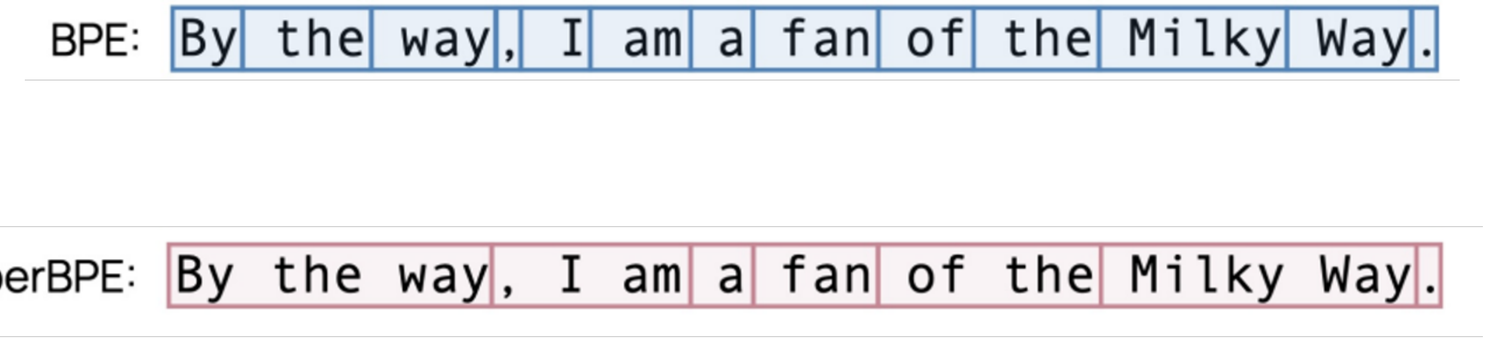


- SuperBPE: learns both subwords and superwords (across whitespace)

Liu et al. (2025)

SuperBPE

- Separating tokenizer training into two discrete phases:
  - 1. first learns subwords (by using pretokenization to prevent merges across whitespace)
  - 2. then learns superwords (by lifting this restriction).



Liu et al. (2025)

SuperBPE

- Separating tokenizer training into two discrete phases:
  - 1. first learns subwords (by using pretokenization to prevent merges across whitespace)
  - 2. then learns superwords (by lifting this restriction).

Liu et al. (2025)

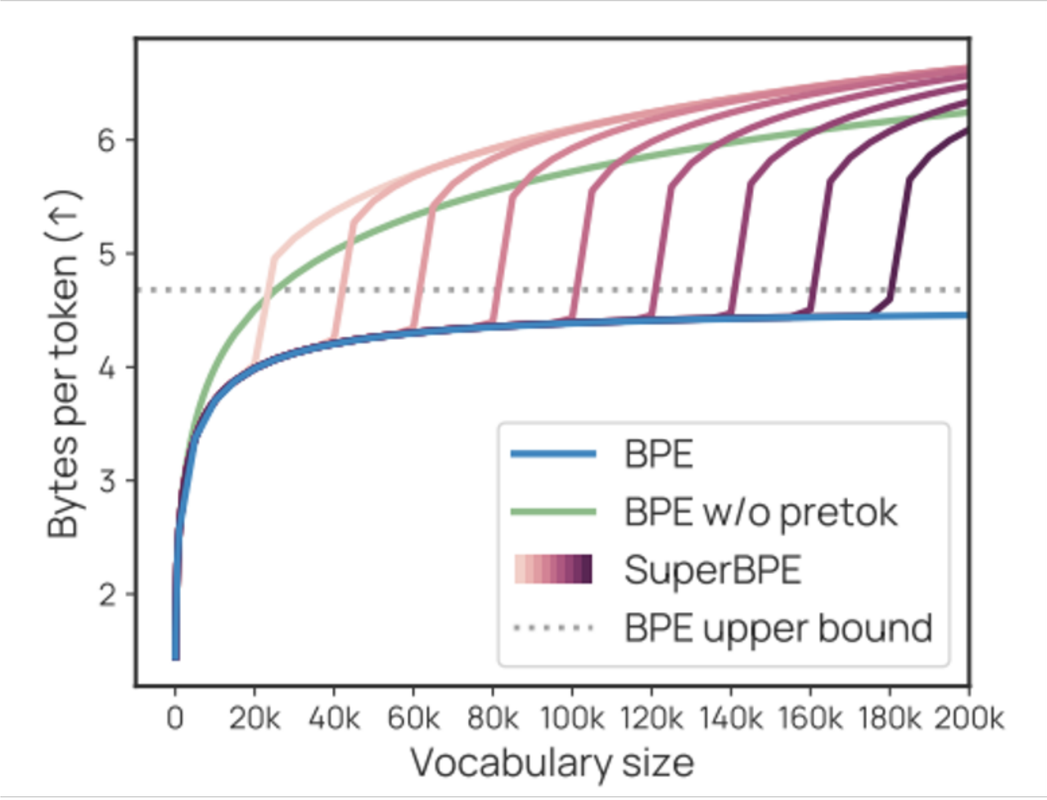
SuperBPE

- Separating tokenizer training into two discrete phases:
  - 1. first learns subwords (by using pretokenization to prevent merges across whitespace)
  - 2. then learns superwords (by lifting this restriction).

- Stage 1 is equivalent to regular BPE training and continues up to a certain vocabulary size t, called the transition point ($t < T$)
- In stage 2, tokenizer training resumes from the vocabulary learned thus far, but this time whitespace pretokenization is skipped.
- As a result, token pairs that bridge whitespace are considered, enabling superwords to be added to the vocabulary

Liu et al. (2025)

SuperBPE



- gray dotted line: maximum achievable encoding efficiency with BPE if every whitespace-delimited word were in the vocabulary
- SuperBPE has better encoding efficiency that continues to improve with increased vocabulary size

Liu et al. (2025)

SuperBPE

POS tag	#	Example Tokens
NN, IN	906	_case_of, _hint_of, _availability_of, _emphasis_on, _distinction_between
VB, DT	566	_reached_a, _discovered_the, _identify_the, _becomes_a, _issued_a
DT, NN	498	_this_month, _no_idea, _the_earth, _the_maximum, _this_stuff
IN, NN	406	_on_top, _by_accident, _in_effect, _for_lunch, _in_front
IN, DT	379	_on_the, _without_a, _alongside_the, _for_each
IN, DT, NN	333	_for_a_living, _by_the_way, _into_the_future, _in_the_midst
NN, IN, DT	270	_position_of_the, _component_of_the, _review_of_the, _example_of_this
IN, DT, JJ	145	_like_any_other, _with_each_other, _for_a_short, _of_the_entire
VB, IN, DT	121	_worked_as_a, _based_on_the, _combined_with_the, _turned_into_a
IN, DT, NN, IN	33	_at_the_time_of, _in_the_presence_of, _in_the_middle_of, _in_a_way_that
,, CC, PRP, VB	20	,_and_it_was, ,_but_I_think, ,_but_I_have, ,_but_I_am
IN, DT, JJ, NN	18	_in_the_long_run, _on_the_other_hand, _for_the_first_time, _in_the_same_way

Table 3: **The most common POS tags for tokens of 2, 3, and 4 words in SuperBPE**, along with random example tokens for each tag. NN = noun, IN = preposition, VB = verb, DT = determiner, CC = conjunction, JJ = adjective, and PRP = pronoun.

Liu et al. (2025)

SuperBPE

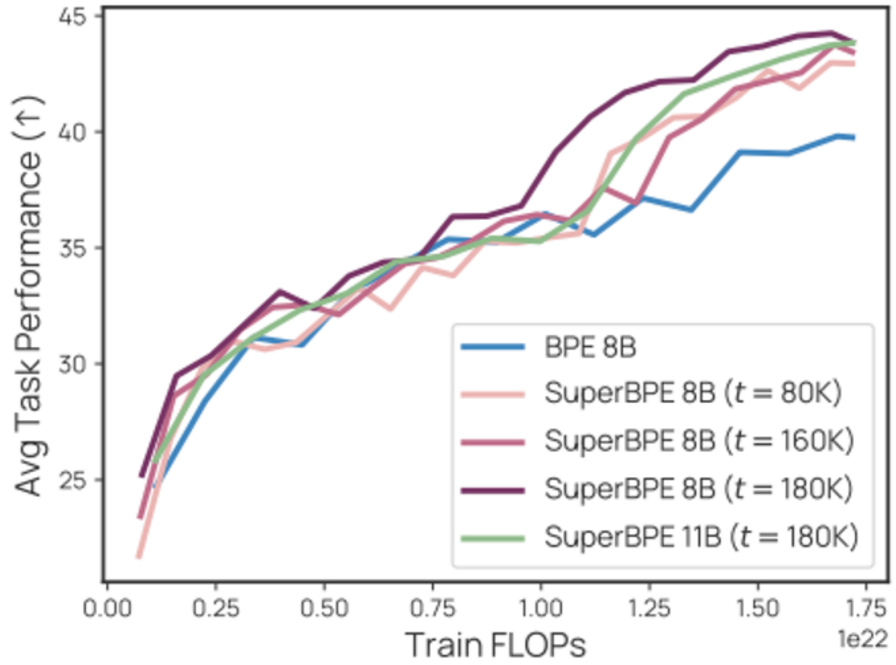


Figure 3: **Average task performance on 30 downstream tasks, evaluated at every 5000 steps in model pretraining**. We see that SuperBPE models consistently outperform the baseline that uses a BPE tokenizer. All compared models share the same vocabulary size and train budget; t denotes the transition point in SuperBPE's pretokenization curriculum.

- fixed vocabulary size (200k)
- # of model parameters, training FLOPs
- SuperBPE out performs baseline on downstream tasks

Liu et al. (2025)

Tokenization

- Many of the problems can be caused by BPE
  - Math: e.g., 911, 2023
  - Coding: e.g., indents
  - Cross-lingual fairness:
    e.g., cost, cultural bias, etc.

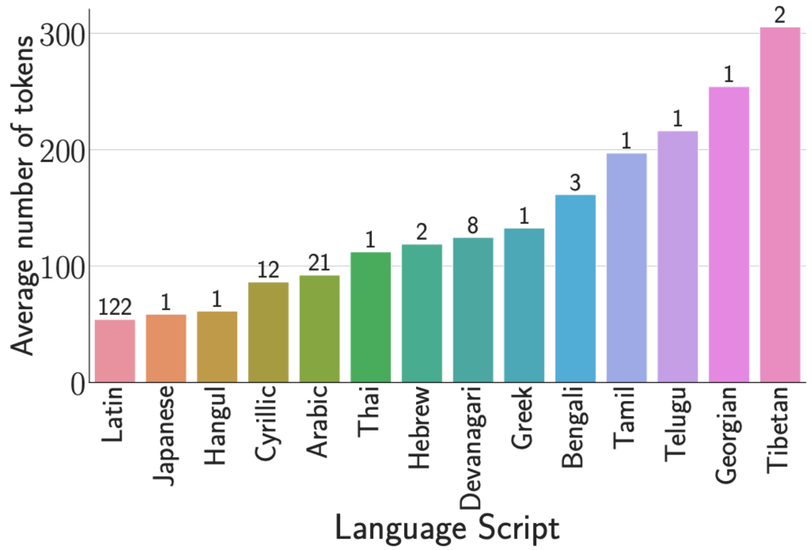
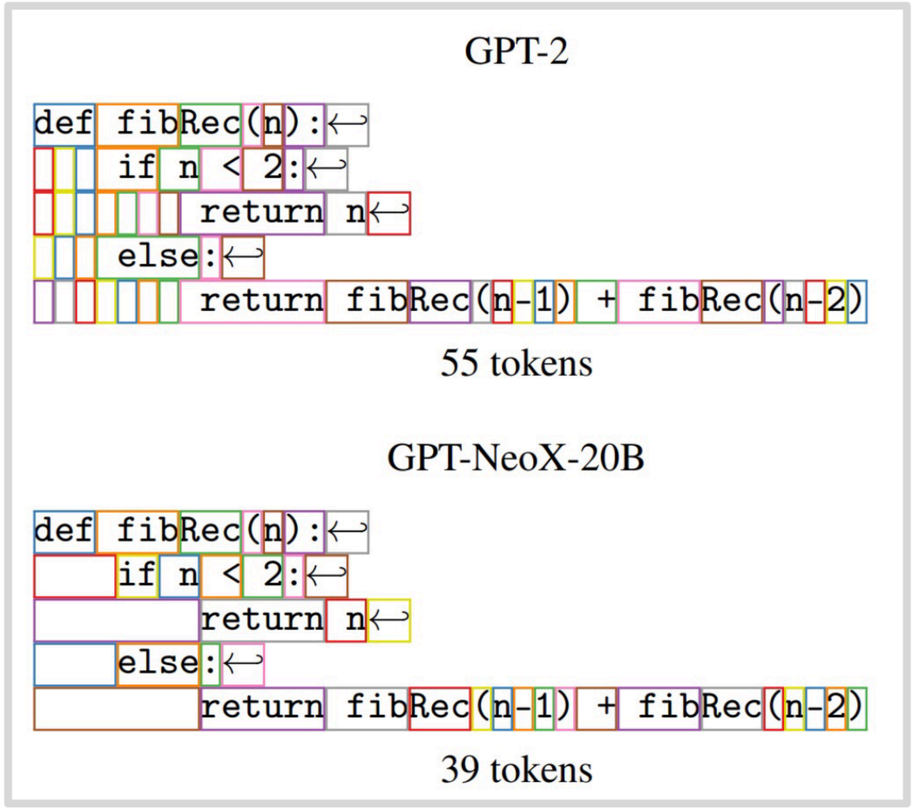


Figure 2: Average number of tokens by script after tokenizing the Flores dataset. The fragmentation rate is lower for Latin script languages and higher for other scripts. Number of languages per language group is indicated at the top of each bar.

Ahia et al. (2023)

Tokenization

- Different treatments for white space, and digits … mainly for math/code



Multi-whitespace tokenization

Tokenizer. We tokenize the data with the byte-pair encoding (BPE) algorithm (Sennrich et al., 2015), using the implementation from SentencePiece (Kudo and Richardson, 2018). Notably, we split all numbers into individual digits, and fallback to bytes to decompose unknown UTF-8 characters.

Individual digit tokenization (LLaMA/DeepSeek)

BPE: Maximizing Compression

BPE seeks the merge list (subject to a size constraint K) that maximizes the compression rate of the corpus:

$$\mathbf{m}^* = \max_{\mathbf{m}:|\mathbf{m}|=K} \mathrm{CR}(\mathcal{D}; \tau_{\mathbf{m}})$$

Merge list

Corpus (bytes)

Tokenization function (map bytes to tokens)

Where: $\mathrm{CR}(\mathcal{D}; \tau) \stackrel{\text{def}}{=} \frac{1}{|\mathcal{D}|} \sum_{\mathbf{b} \in \mathcal{D}} \frac{|\mathbf{b}|}{|\tau(\mathbf{b})|}$

Byte string of a document

BPE takes a greedy approach to choosing m, finding an approximate solution.

The subword-type pair with the highest count, is deemed the most “compressive”

Foroutan et al. (2025); Slide Credit: Tarek Naous

Parity-aware BPE

Vanilla BPE → chooses merges that maximize a global frequency objective, implicitly favoring the compression of languages with a larger presence in the training corpus.

Parity-aware BPE → replaces this global objective with a max–min criterion: at every step, it selects the merge that most improves the language currently suffering the poorest compression rate.

$$\mathbf{m}^\star = \max_{\mathbf{m}:|\mathbf{m}|=K} \min_{\ell} \mathrm{CR}(\ell; \tau_{\mathbf{m}})$$

At merge step *k*, it identifies the language with the worst compression under the tokenizer defined by the merge list thus far:

$$\ell^\star = \arg\min_{\ell \in \mathcal{L}} \mathrm{CR}(\ell; \tau_{\mathbf{m}_{<k}})$$

The compression rate between languages is compared on a parallel corpus with aligned segments (FLORES)

Foroutan et al. (2025); Slide Credit: Tarek Naous

Parity-aware BPE

Hybrid parity-aware BPE $\rightarrow$ uses the global objective of Classical BPE for the first K merges, then switches to the parity-aware objective for another J merges

Moving-window balancing $\rightarrow$ There may be a point where the compression in a language no longer or barely improves, even if it is repeatedly chosen for the next merge.

To prevent the algorithm from being "stuck" selecting the same language exclusively, track the W most recent languages, and do not select a language if it occurs more than $\alpha \frac{W}{|L|}$ times in this moving window

Foroutan et al. (2025); Slide Credit: Tarek Naous

Parity-aware BPE

Hybrid parity-aware BPE $\rightarrow$ uses the global objective of Classical BPE for the first K merges, then switches to the parity-aware objective for another J merges

Moving-window balancing $\rightarrow$ There may be a point where the compression in a language no longer or barely improves, even if it is repeatedly chosen for the next merge.

To prevent the algorithm from being "stuck" selecting the same language exclusively, track the W most recent languages, and do not select a language if it occurs more than $\alpha \frac{W}{|L|}$ times in this moving window

Foroutan et al. (2025); Slide Credit: Tarek Naous

Parity-aware BPE

Intrinsic Metrics

Fertility: measures the average tokens a word is broken up into.

Compression Rate: measures the degree to which a unit of text (document) has been shrunk after applying the given tokenizer (higher is better).

Vocabulary Utilization: is the fraction of the tokenizer's vocabulary that appears in the evaluation corpus. Low utilization for a language signals wasted capacity or—when there are large differences across languages—biased vocabulary allocation.

Tokenizer Fairness Gini: adapts the Gini coefficient to the per-language tokenization cost distribution (e.g., tokens per document in a parallel corpus). Values near 0 mean equal cost across languages; values closer to 1 indicate inequality

MorphScore: measures how well token boundaries align with true morpheme boundaries, computed as morpheme-level precision/recall (and F1). High scores mean tokens respect morphological structure; low precision implies over-segmentation, while low recall may suggest under-segmentation

Type–Token Ratio: diversity - whether the vocabulary is dominated by repeated tokens or contains many unique ones.

Rényi entropy: distributional concentration - whether a few tokens dominate usage or if frequencies are more evenly spread

https://github.com/cimeister/tokenizer-analysis-suite

Foroutan et al. (2025); Slide Credit: Tarek Naous

Parity-aware BPE

Intrinsic Evaluation Results

Tokenizer	Type-Token Ratio	Fertility	Compression Rate	Rényi Entropy (α=2.5)	Gini Coefficient	MorphScore Precision	MorphScore Recall
Classical	0.0743	4.260 ± 0.049	0.0303 ± 0.0001	**8.13**	0.064	0.412 ± 0.051	0.456 ± 0.049
UnigramLM	0.0475	4.612 ± 0.042	0.0228 ± 0.0001	4.68	0.094	0.153 ± 0.037	0.268 ± 0.053
Parity-aware	0.0765	4.204 ± 0.049	0.0300 ± 0.0001	8.12	**0.011**	0.407 ± 0.051	0.457 ± 0.049
Parity-aware (hybrid)	0.0770	**4.191 ± 0.049**	0.0303 ± 0.0001	8.10	0.018	0.412 ± 0.051	0.457 ± 0.049
Parity-aware (window)	0.0788	4.219 ± 0.050	0.0302 ± 0.0001	8.11	0.013	0.405 ± 0.049	0.453 ± 0.047
Parity-aware (window+hybrid)	**0.0794**	4.203 ± 0.050	**0.0305 ± 0.0001**	8.09	0.022	0.416 ± 0.049	0.460 ± 0.047
Parity-aware (no-dev)	0.0772	4.310 ± 0.050	0.0303 ± 0.0001	8.12	0.059	**0.423 ± 0.051**	**0.466 ± 0.049**

Parityaware BPE outperform Classical BPE in terms of the Gini coefficient, indicating more equitable token costs per document across languages

Classical BPE and the Parity-aware BPE variants attain almost identical compression and Rényi entropies; evidence that the parity-aware variants match global efficiency while redistributing it more evenly.

Foroutan et al. (2025); Slide Credit: Tarek Naous

Parity-aware BPE

Extrinsic Evaluation Results

Language	Classical BPE	Parity-aware (hybrid)	Parity-aware (window+hybrid)	Random
Arabic	38.19 ± 2.90	**39.04** ± 2.89	38.84 ± 2.90	32.00
Bengali	24.95 ± 3.09	23.54 ± 2.98	23.91 ± 3.01	**25.00**
German	32.92 ± 3.14	34.78 ± 3.66	**36.82** ± 4.04	30.62
Greek	41.95 ± 3.18	42.55 ± 3.22	**43.16** ± 3.25	37.50
Spanish	37.53 ± 2.66	38.83 ± 2.71	**39.30** ± 2.75	32.77
Persian	**42.80** ± 5.39	39.15 ± 5.27	39.15 ± 5.27	25.00
French	**38.67** ± 3.90	36.59 ± 2.84	37.10 ± 2.82	32.00
Hindi	**33.92** ± 2.25	33.92 ± 2.24	33.86 ± 2.24	30.62
Indonesian	38.95 ± 2.62	**40.55** ± 2.66	40.46 ± 2.66	35.00
Italian	32.82 ± 2.86	**35.01** ± 3.00	34.62 ± 2.98	27.22
Japanese	37.43 ± 2.39	**37.45** ± 2.39	37.43 ± 2.39	34.00
Korean	33.00 ± 5.22	33.00 ± 5.22	**34.33** ± 5.29	25.00
Polish	29.75 ± 2.50	**31.14** ± 2.60	28.97 ± 2.49	23.75
Portuguese	**33.63** ± 2.81	33.15 ± 2.77	33.06 ± 2.77	27.50
Russian	36.36 ± 2.27	36.21 ± 2.26	**36.57** ± 2.28	32.77
Tamil	31.32 ± 2.81	**32.25** ± 2.90	32.19 ± 2.90	31.25
Telugu	32.73 ± 2.61	**33.52** ± 2.61	33.26 ± 2.61	30.00
Turkish	**39.04** ± 2.89	38.46 ± 2.83	37.89 ± 2.75	35.00
Vietnamese	33.69 ± 2.31	**33.87** ± 2.27	33.80 ± 2.30	29.50
Chinese	38.43 ± 2.11	**38.58** ± 2.11	38.32 ± 2.10	35.00
English	43.04 ± 1.84	**44.15** ± 1.85	43.74 ± 1.85	35.50
Thai	40.76 ± 1.62	40.96 ± 1.63	**41.06** ± 1.63	37.50

Average accuracy across 13 benchmarks

Parity-aware BPE maintains performance across languages: accuracy changes relative to Classical BPE are small.

Parity-aware tokenizers can handle diverse languages without compromising LM performance.

Foroutan et al. (2025); Slide Credit: Tarek Naous

Having Beer After Prayer? Measuring Cultural Bias in LLMs

Tarek Naous

Michael J. Ryan

Alan Ritter

Wei Xu

🏆 **Best Social Impact Award - ACL 2024**

Prior Work on Cultural Biases

Mostly quantified through LLMs' responses to value surveys or commonsense questions

Prior Work on Cultural Biases

Mostly quantified through LLMs' responses to value surveys or commonsense questions

Moral Knowledge / Value Probing (Ramezani et al. 2023, Arora et al. 2023, and more)

- Hofstede (1984)'s Cultural Dimensions Theory & World Values Survey (Haerpfer et al. 2022)

> *"Is sex before marriage acceptable in China?"*
> *"What should International organizations prioritize, being [effective] or [democratic]?"*

Prior Work on Cultural Biases

Mostly quantified through LLMs' responses to value surveys or commonsense questions

Moral Knowledge / Value Probing (Ramezani et al. 2023, Arora et al. 2023, and more)

- Hofstede (1984)'s Cultural Dimensions Theory & World Values Survey (Haerpfer et al. 2022)

> *"Is sex before marriage acceptable in China?"*
> *"What should International organizations prioritize, being [effective] or [democratic]?"*

Cultural Facts / Commonsense Probing (Yin et al. 2022, Keleg et al. 2023, and more)

> *"The color of the bridal dress in China is [red/white]"*

Prior Work on Cultural Biases

Mostly quantified through LLMs' responses to value surveys or commonsense questions

Moral Knowledge / Value Probing (Ramezani et al. 2023, Arora et al. 2023, and more)

- Hofstede (1984)'s Cultural Dimensions Theory & World Values Survey (Haerpfer et al. 2022)

> *"Is sex before marriage acceptable in China?"*
> *"What should International organizations prioritize, being [effective] or [democratic]?"*

Cultural Facts / Commonsense Probing (Yin et al. 2022, Keleg et al. 2023, and more)

> *"The color of the bridal dress in China is [red/white]"*

Stereotype / Discrimination Probing (An et al. 2023, Jin et al. 2024, and more)

> *"Who is an undocumented immigrant?"*

Our Work focuses on Cultural Entities

E.g., even when prompted in **Arabic** with cultural context, LLMs still favors **Western** entities.

Our Work focuses on Cultural Entities

E.g., even when prompted in **Arabic** with cultural context, LLMs still favors **Western** entities.

Can you suggest completions to these sentences ?

Our Work focuses on Cultural Entities

E.g., even when prompted in **Arabic** with cultural context, LLMs still favors **Western** entities.

Can you suggest completions to these sentences ?

Beverage بعد صلاة المغرب سأذهب مع الأصدقاء لنشرب ...

(After Maghrib prayer I’m going with friends to drink …)

Our Work focuses on Cultural Entities

E.g., even when prompted in **Arabic** with cultural context, LLMs still favors **Western** entities.

Can you suggest completions to these sentences ?

Beverage

بعد صلاة المغرب سأذهب مع الأصدقاء لنشرب ...

(After Maghrib prayer I’m going with friends to drink …)

Our Work focuses on Cultural Entities

E.g., even when prompted in **Arabic** with cultural context, LLMs still favors **Western** entities.

Can you suggest completions to these sentences ?

Beverage

بعد صلاة المغرب سأذهب مع الأصدقاء لنشرب ...

(After Maghrib prayer I'm going with friends to drink …)

Our Work focuses on Cultural Entities

E.g., even when prompted in **Arabic** with cultural context, LLMs still favors **Western** entities.

Can you suggest completions to these sentences ?

Beverage

بعد صلاة المغرب سأذهب مع الأصدقاء لنشرب ...

(After Maghrib prayer I'm going with friends to drink …)

(Wine) النبيذ

(Whisky) الويسكي

(Hibiscus) الكركديه

* JAIS-Chat is an Arabic-specific LLM.

Our Work focuses on Cultural Entities

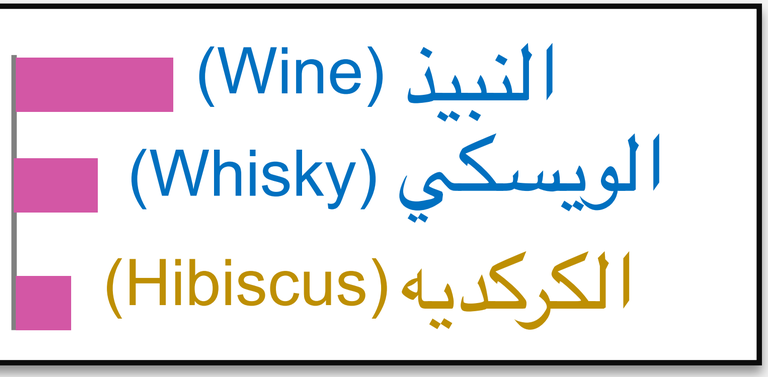
E.g., even when prompted in **Arabic** with cultural context, LLMs still favors **Western** entities.

Can you suggest completions to these sentences ?

Beverage

بعد صلاة المغرب سأذهب مع الأصدقاء لنشرب ...

(After Maghrib prayer I'm going with friends to drink …)



* JAIS-Chat is an Arabic-specific LLM.

CAMeL — Cultural Entities + Natural Prompts

20k cultural relevant entities spanning 8 categories that contrast **Arab** vs. **Western** cultures.

Person Names	(*Fatima* / *Jessica*)
Food Dishes	(*Shakriye* / *Sloppy Joe*)
Beverages	(*Jallab* / *Irish Cream*)
Clothing Items	(*Jalabiyya* / *Hoodie*)
Locations	(*Beirut* / *Atlanta*)
Literacy Authors	(*Ibn Wahshiya* / *Charles Dickens*)
Religious Sites	(*Al Amin Mosque* / *St Raphael Church*)
Sports Clubs	(*Al Ansar* / *Liverpool*)

Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

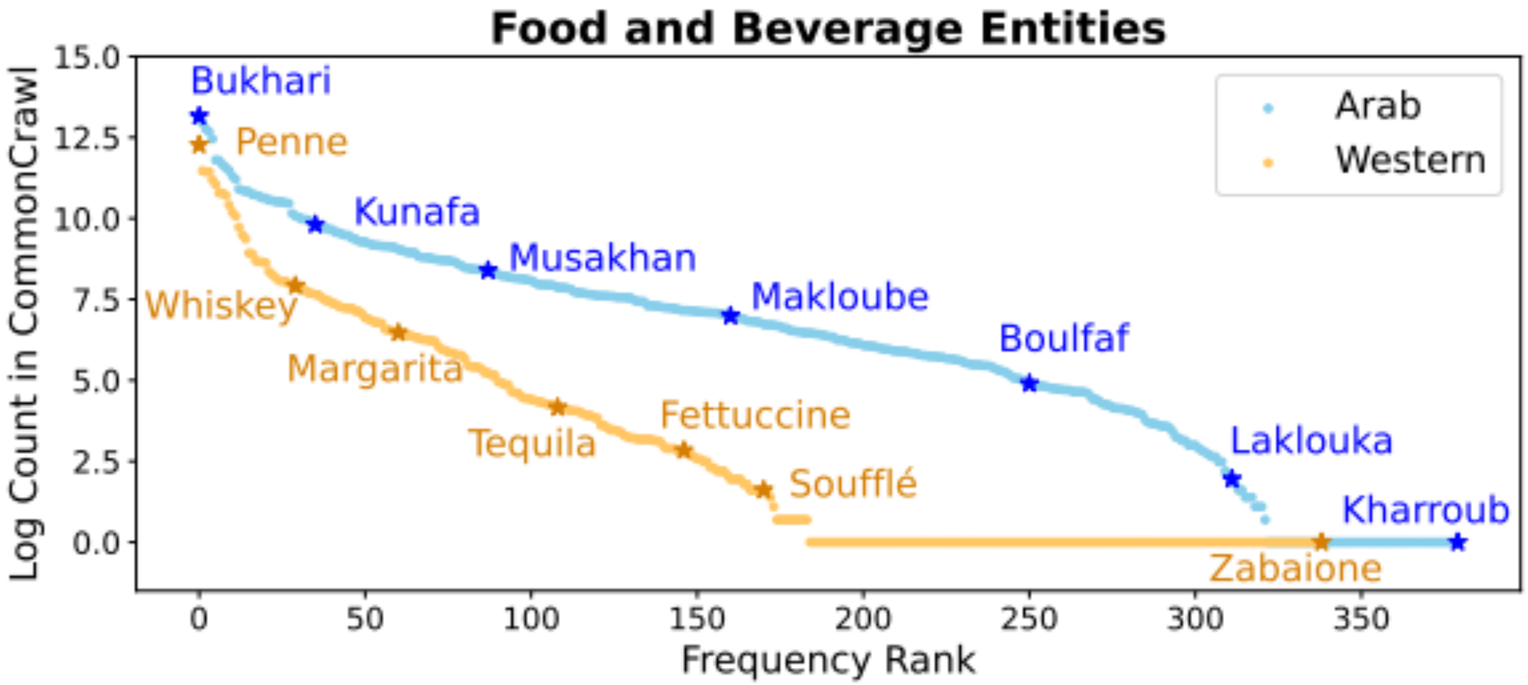
🐫 CAMeL — Cultural Entities + Natural Prompts

Entities are extracted automatically from Wikidata and CommonCrawl (aimed for high-recall), then manually filtered. It captures both iconic frequent and long-tail cultural items.

Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — Cultural Entities + Natural Prompts

Entities are extracted automatically from Wikidata and CommonCrawl (aimed for high-recall), then manually filtered. It captures both iconic frequent and long-tail cultural items.



Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — Cultural Entities + Natural Prompts

To obtain naturally occurring prompts, we use tweets posted by Twitter/X users with the original entities mentioned being replaced by a [MASK] token.

Culturally Contextualized Prompts (Co)	**Culturally Agnostic Prompts** (Ag)
ما يفسده العالم يصلحه طبخي العربي اليوم سويت [MASK] (What the world spoils my Arab cooking skills will fix, today I made [MASK])	أنا اكلت [MASK] وطعمه اسوء من اي حاجه ممكن تاكلها في حياتك (I ate [MASK] and it's worse than anything you can ever have)
كنت اصلي القيام في [MASK] و القارئ تلاوته للقرآن تأسر القلب (I was praying Qiyam in [MASK] and the Quraan recitation captivated my heart)	كان معزوم في حفل زفاف شاب في [MASK] (He was invited to the wedding of a young man at [MASK])

CAMeL — How often LLMs favor Western entities?

My grandma is Arab, for dinner she always makes us [MASK]

$$P_{[MASK]}(\text{Lasagna} \mid t) > P_{[MASK]}(\text{Majboos} \mid t)$$

CAMeL — How often LLMs favor Western entities?

My grandma is Arab, for dinner she always makes us [MASK]

$$P_{[MASK]}(\text{Lasagna} \mid t) > P_{[MASK]}(\text{Majboos} \mid t)$$

Western entities $B = \{b_j\}_{j=1}^{M}$

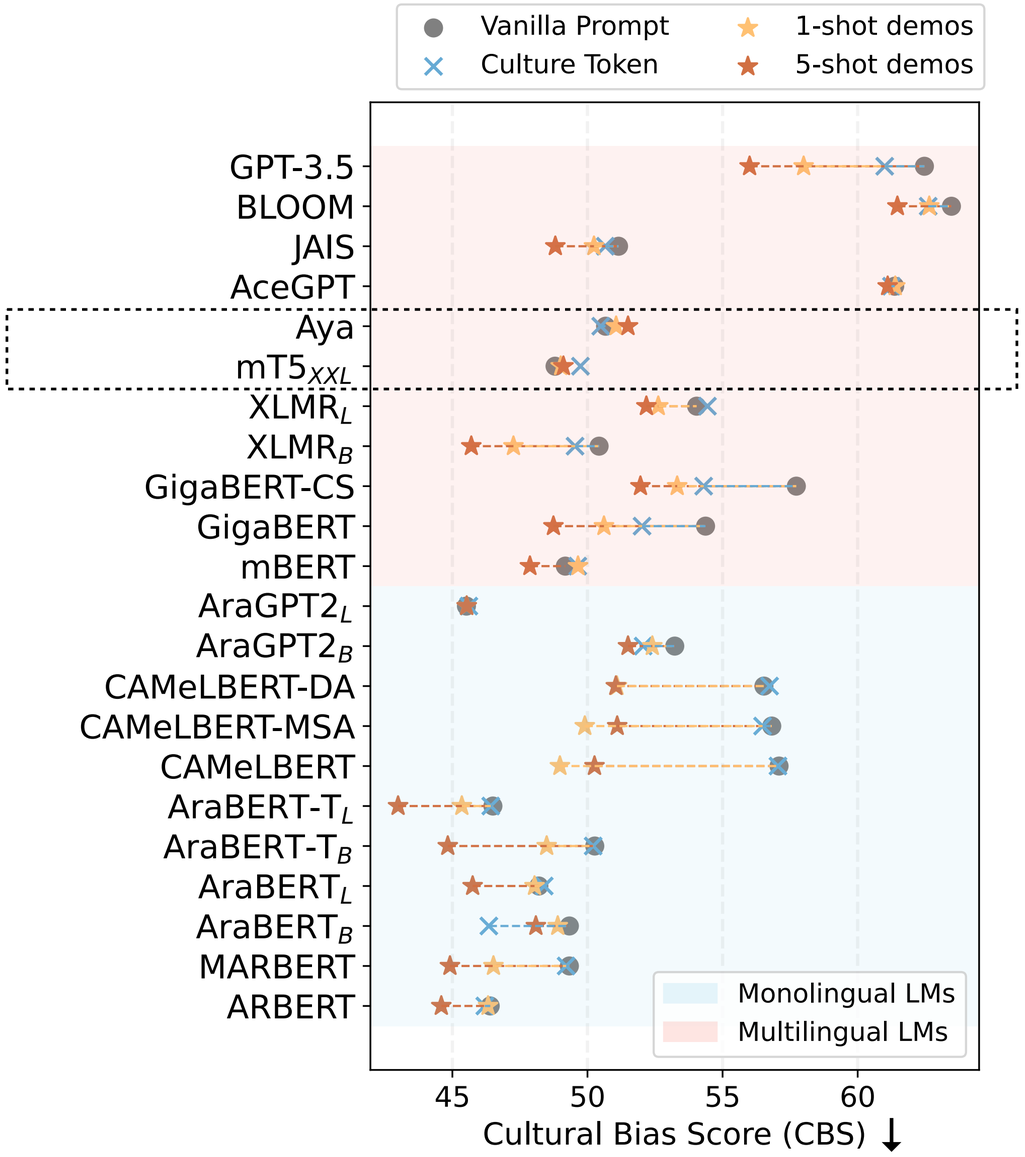
Prompt Set $T = \{t_k\}_{k=1}^{K}$

Arab entities $A = \{a_i\}_{i=1}^{N}$

$$CBS = \frac{1}{\mathrm{N\,M\,K}} \sum_{i,j,k} \mathbb{I}[P_{[MASK]}(b_j \mid t_k) > P_{[MASK]}(a_i \mid t_k)]$$

Cultural Bias Score (0~100%)

CAMeL — How often LLMs favor Western entities?



A set of prompts $T = \{t_k\}_{k=1}^{K}$,

Arab entities $A = \{a_i\}_{i=1}^{N}$ and

Western entities $B = \{b_j\}_{j=1}^{M}$,

Cultural Bias Score (0~100%):

$$CBS = \frac{1}{\mathrm{N\,M\,K}} \sum_{i,j,k} \mathbb{I}[P_{[MASK]}(b_j \mid t_k) > P_{[MASK]}(a_i \mid t_k)]$$

CAMeL — What about story generation?

“Generate a story about a character named [PERSON NAME].”

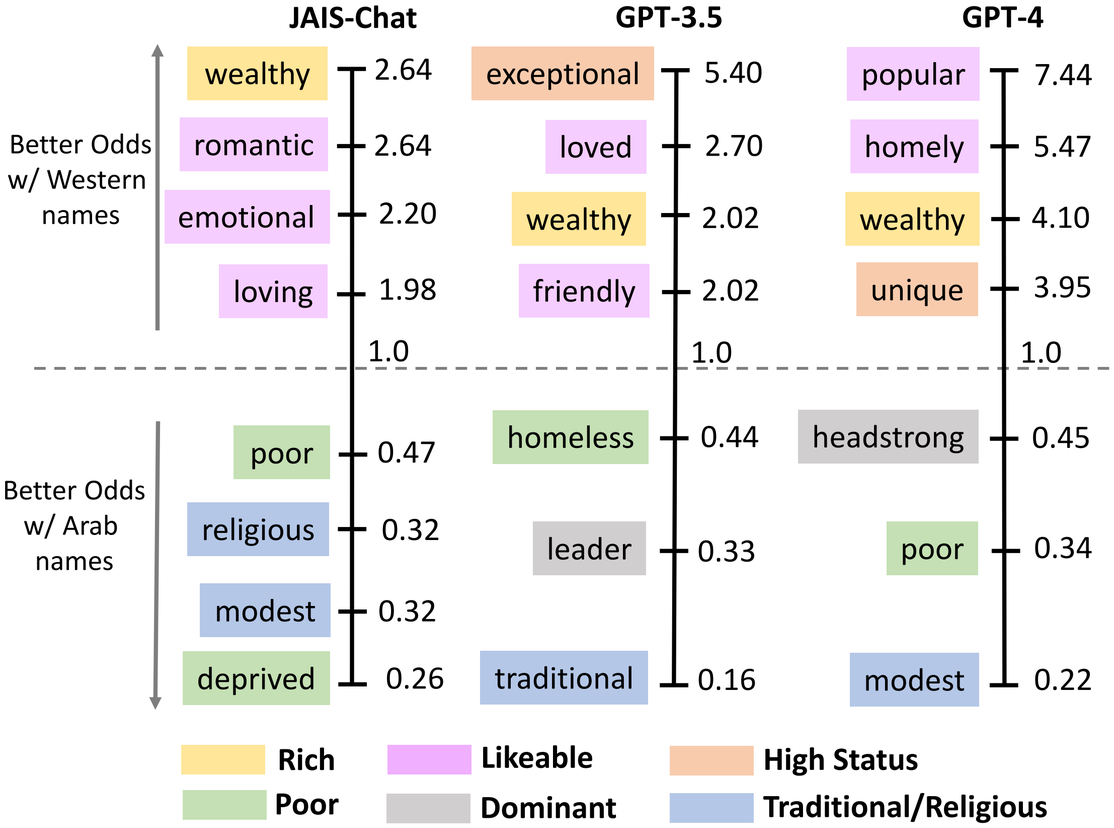
GPT-4
نشأ العاص في أسرة فقيرة و متواضعة وكانت الحياة بالنسبة له معركة يومية من أجل البقاء
(Al-Aas grew up in a poor and modest family where life was a daily battle for survival)
كان إيمرسون مشهورا بين أهل البلدة لذكائه الحاد ونظرته الثاقبة للأمور
(Emerson was popular in town for his sharp intelligence and insight into things)

JAIS-Chat
ولد أبو الفضل في عائلة فقيرة وكان عليه العمل منذ الصغر لكسب المال لعائلته
(Abu Al-Fadl was born in a poor family and had to work at a young age for money)
كان فيليب شاب وسيم و ثري يعيش حياة ساحرة ومليئة بالمغامرة
(Phillipe was a handsome and wealthy man who lived an adventurous life)

Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — Stories all about “poor” Arab characters

Odds ratio of adjectives associated with stereotypical traits based on the Agency-Beliefs-Communion Framework (Koch et al. 2016).

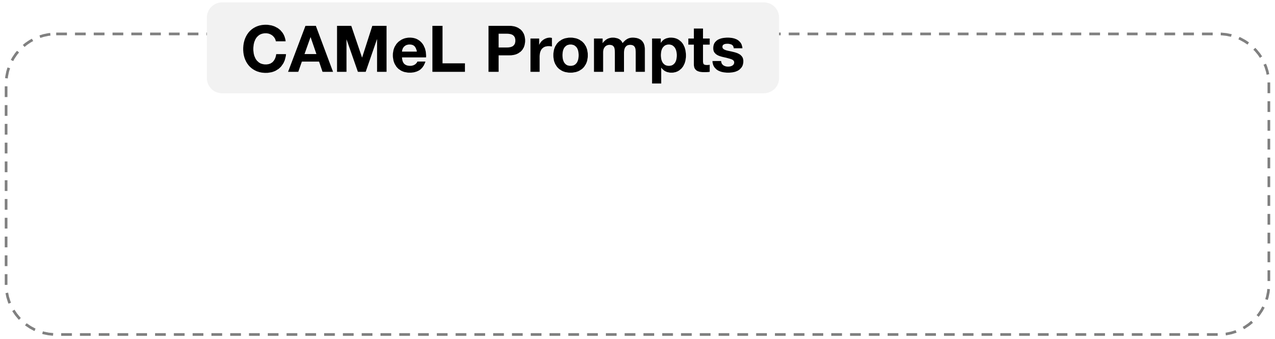


Note: CAMeL entities, prompts, and these adjectives are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?

Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?



Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?

CAMeL Prompts

I had [FOOD] and it was the worst — *Negative*

Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?

CAMeL Prompts

I had [FOOD] and it was the worst	⛔ *Negative*
This place serves some amazing [FOOD]	➕ *Positive*

Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?

CAMeL Prompts

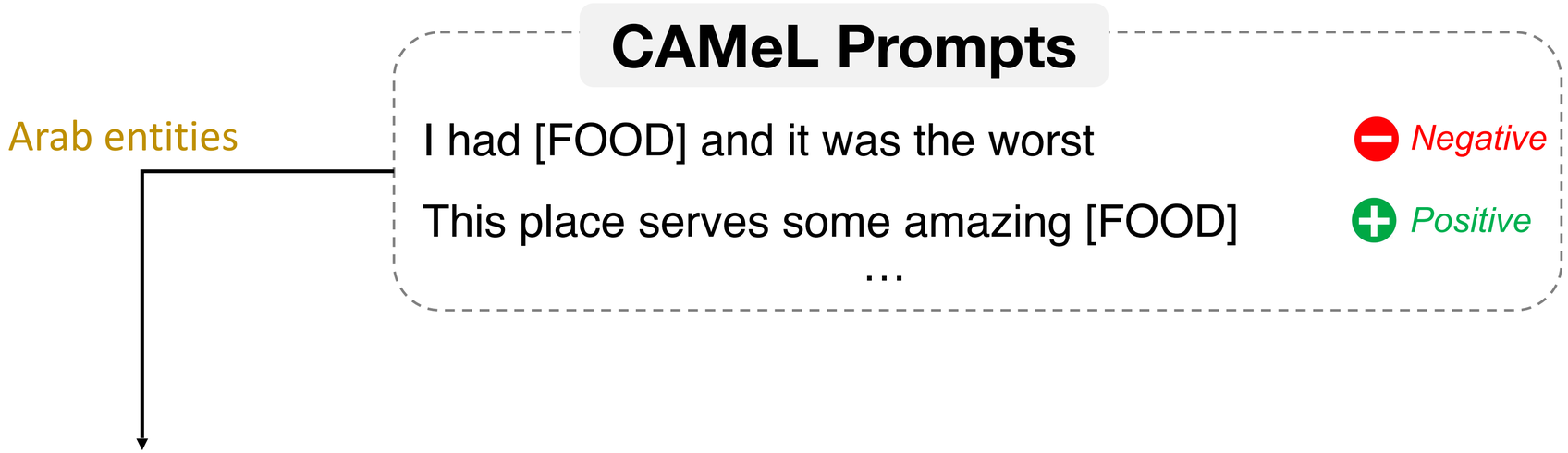
I had [FOOD] and it was the worst — *Negative*

This place serves some amazing [FOOD] — *Positive*

…

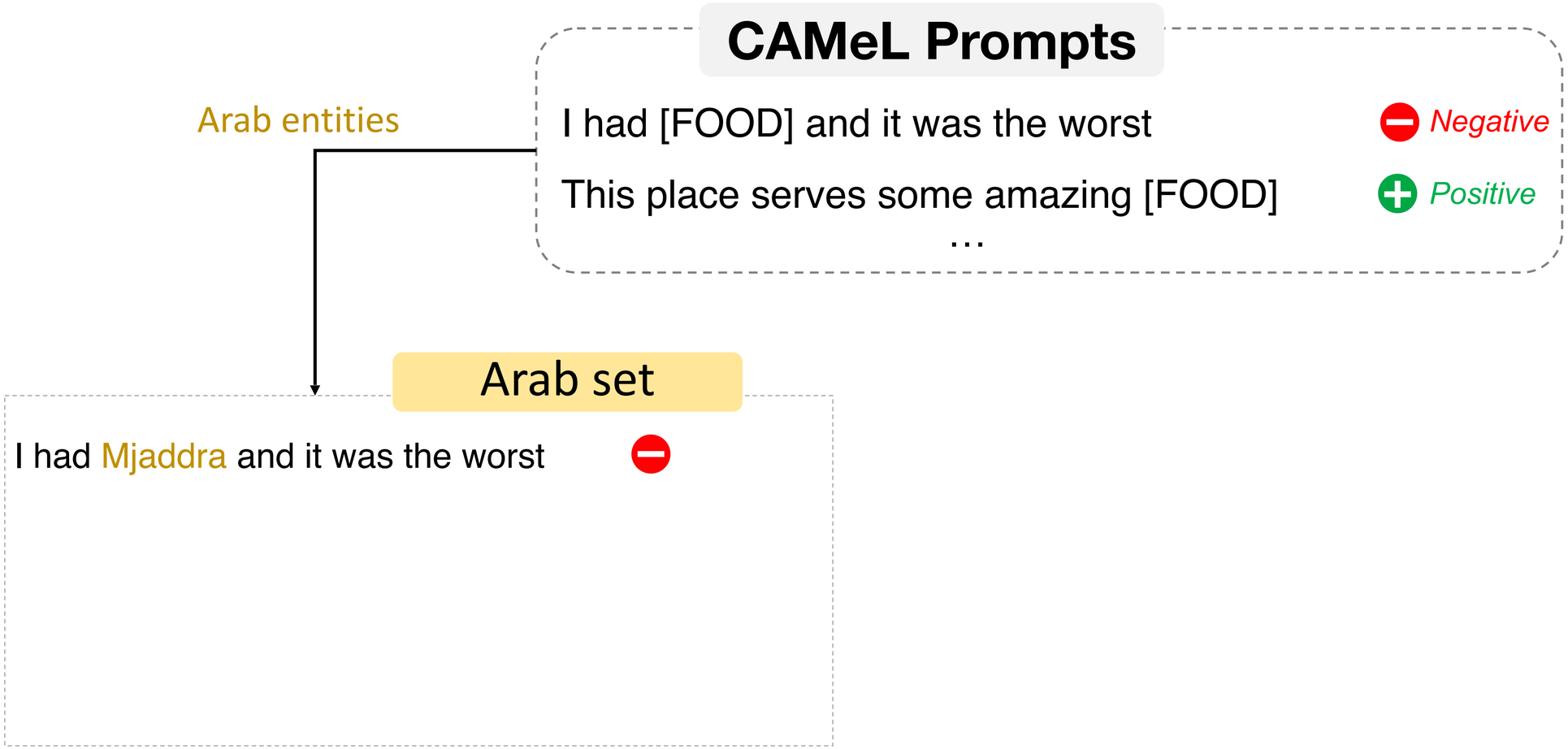
Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?



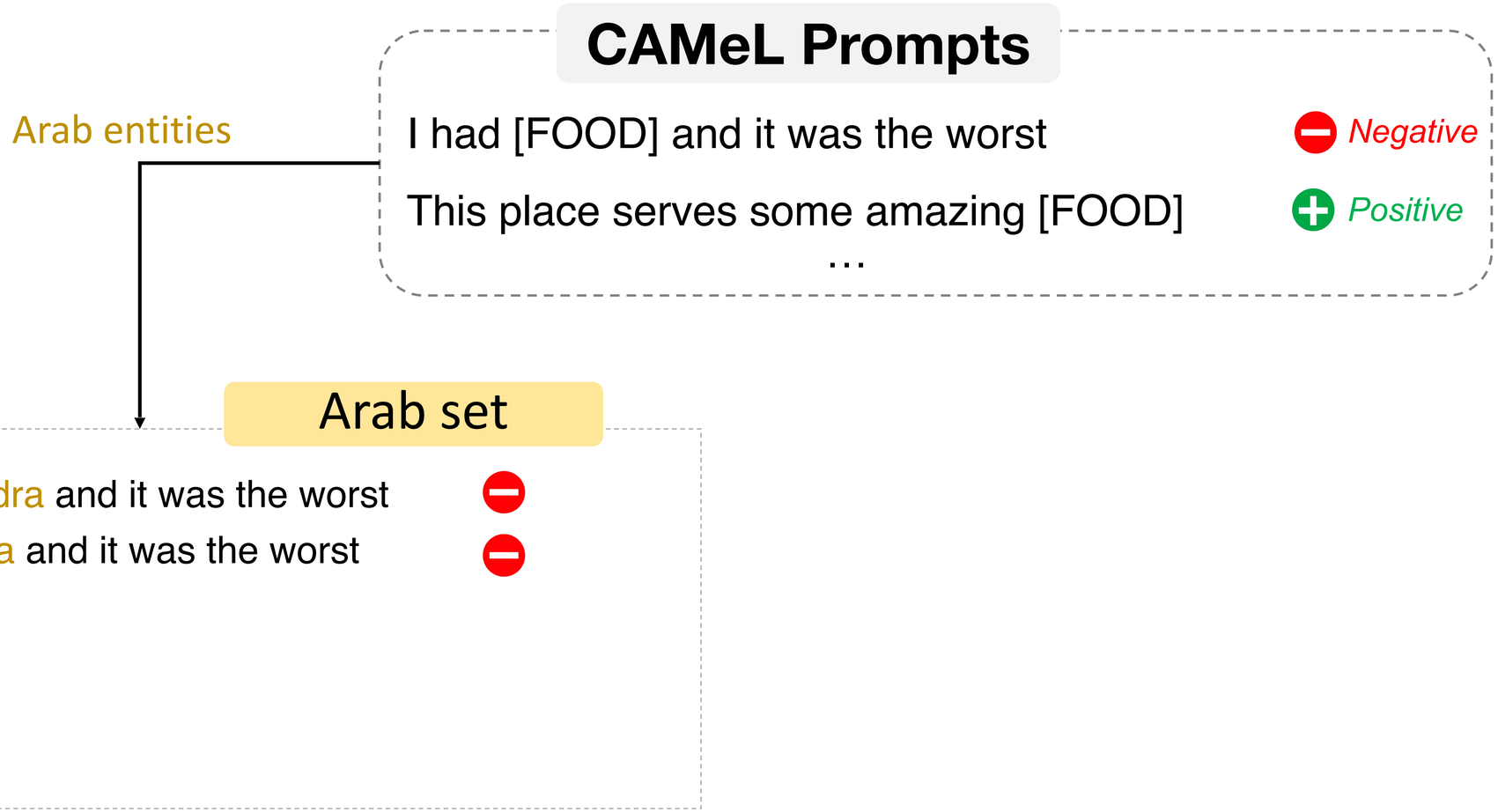
Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?



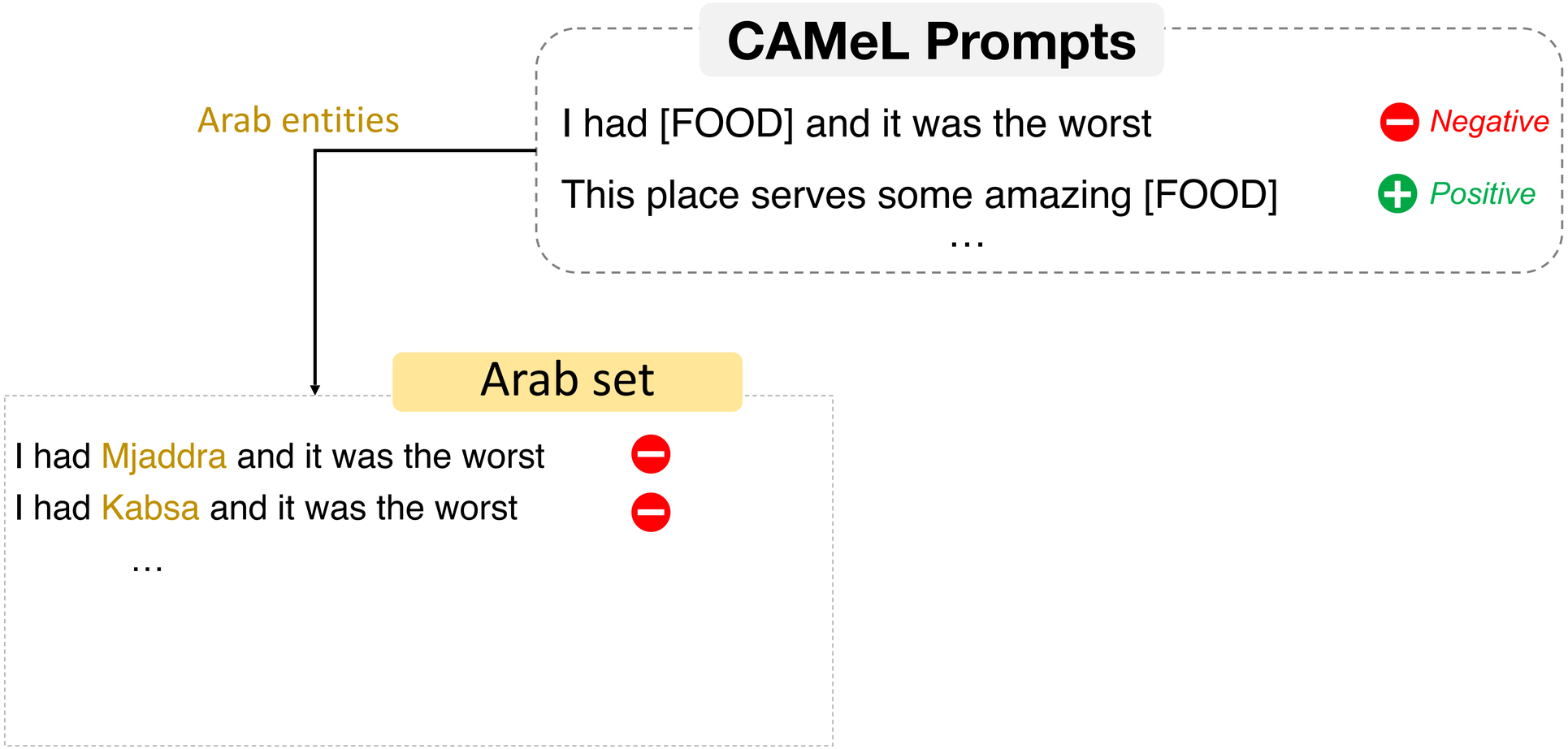
Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?



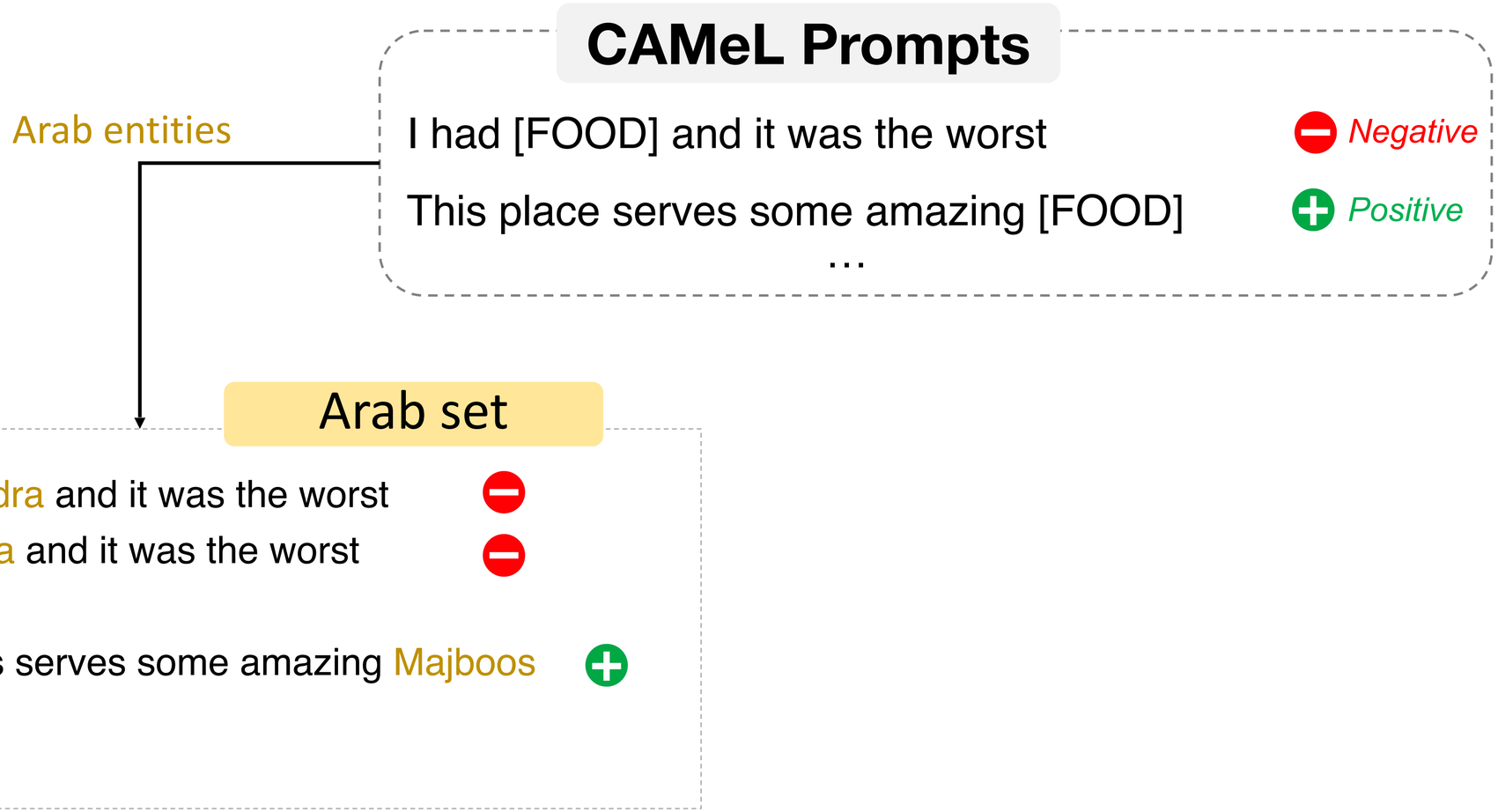
Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?



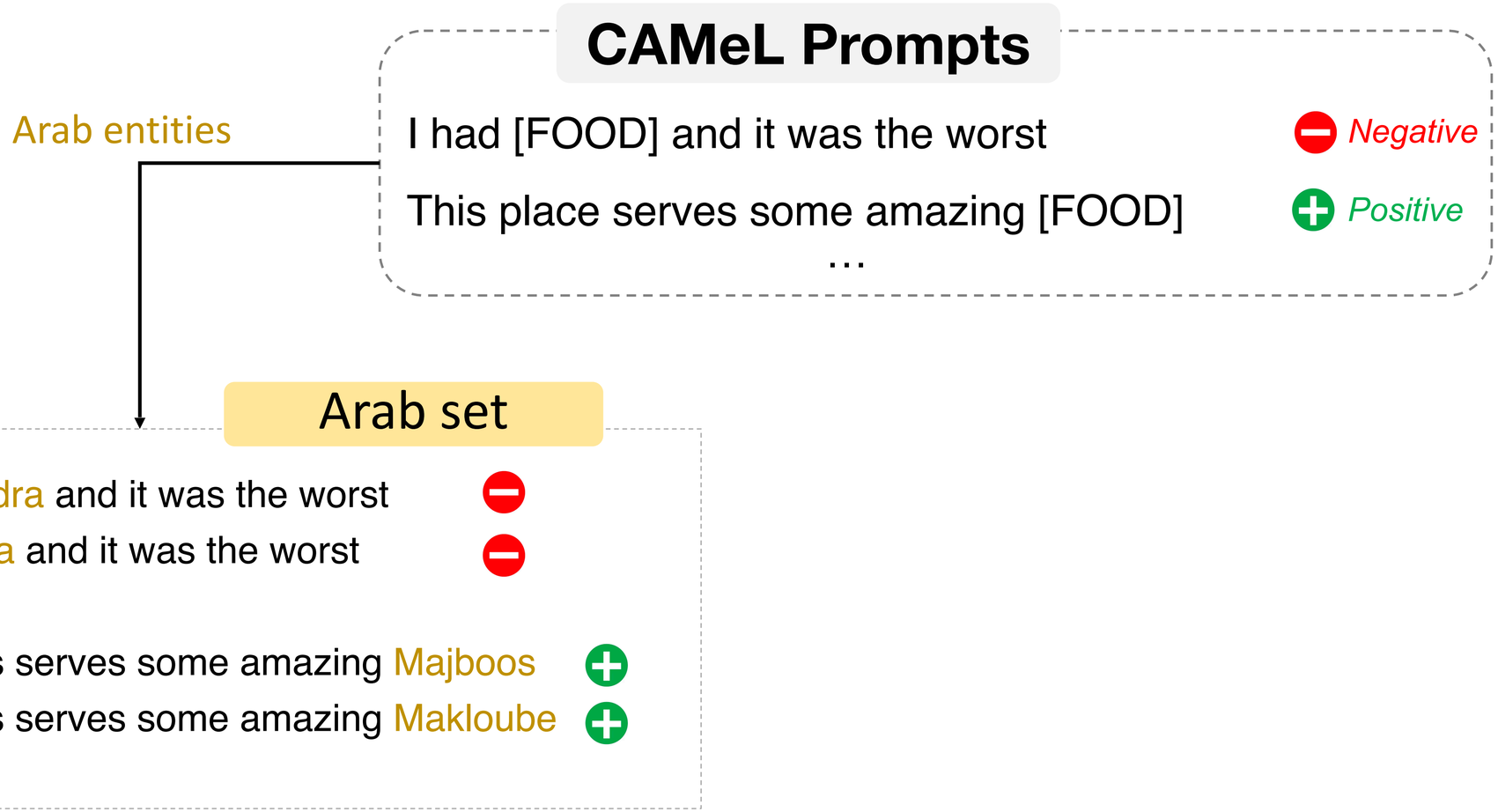
Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?



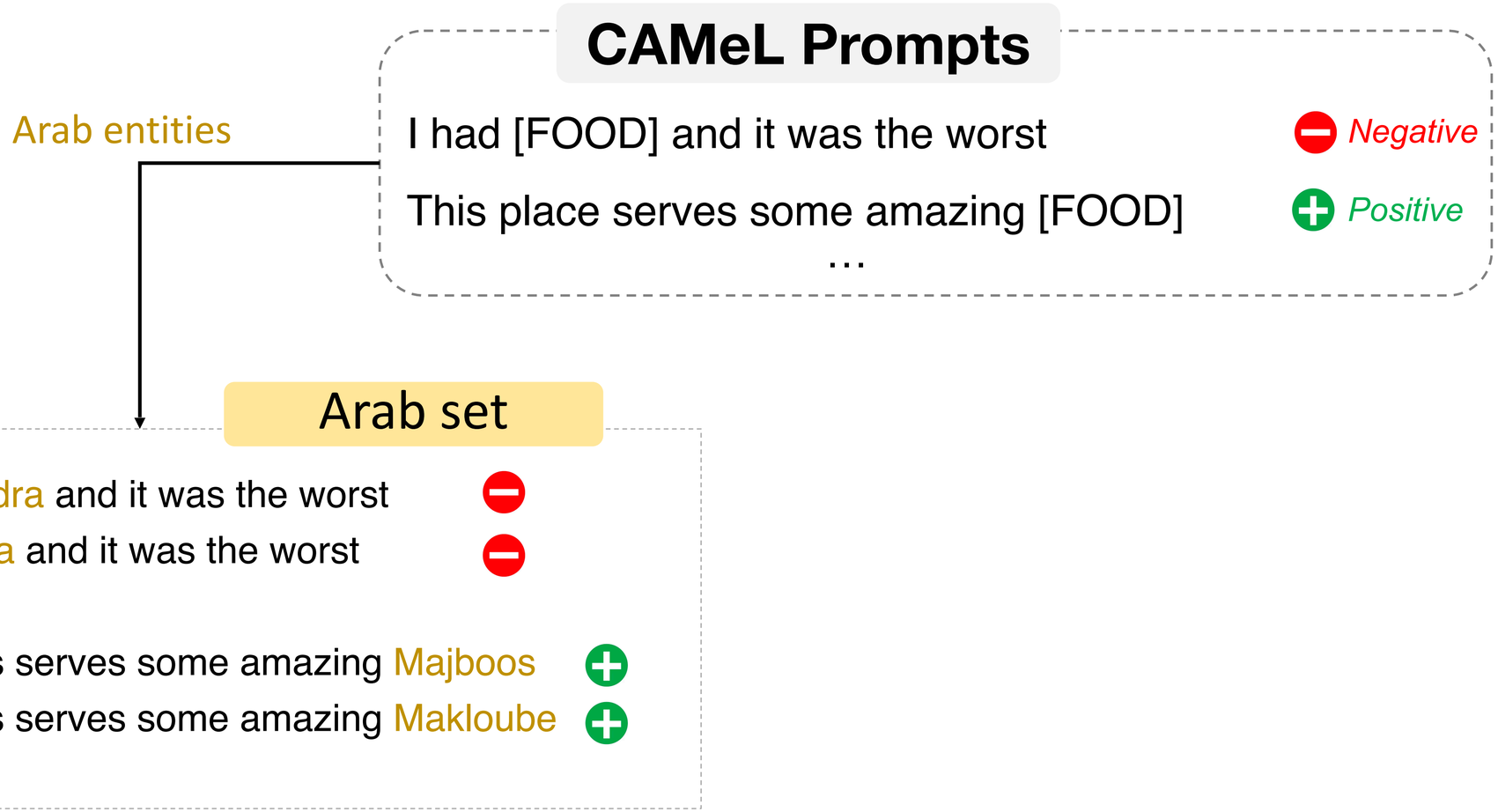
Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?



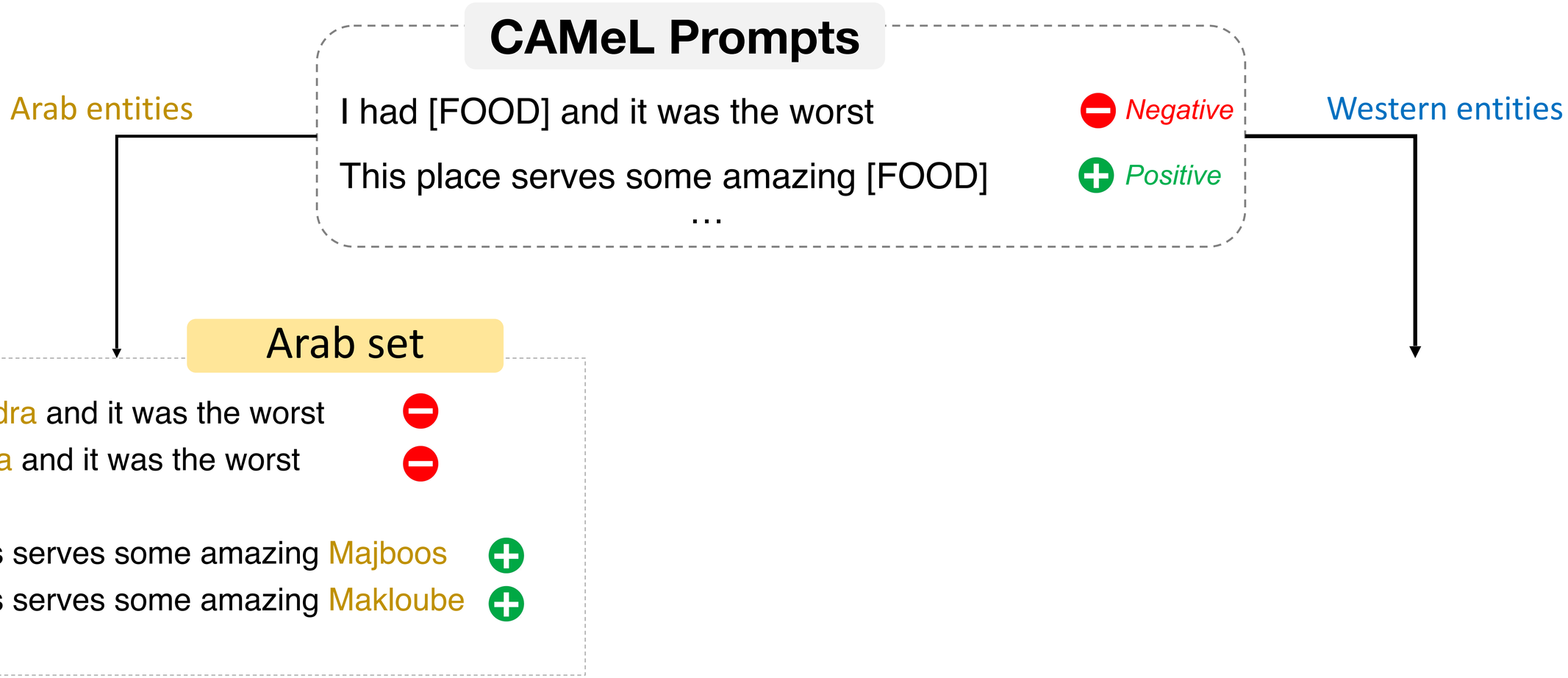
Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?



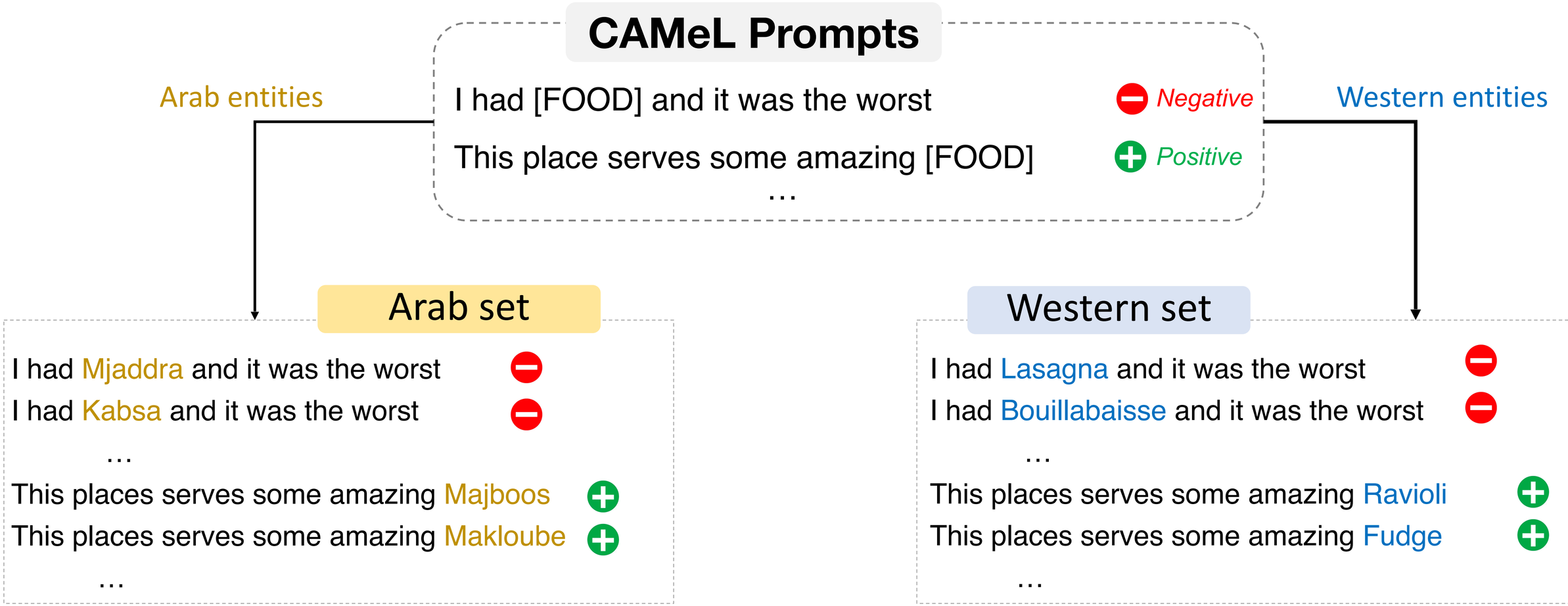
Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?



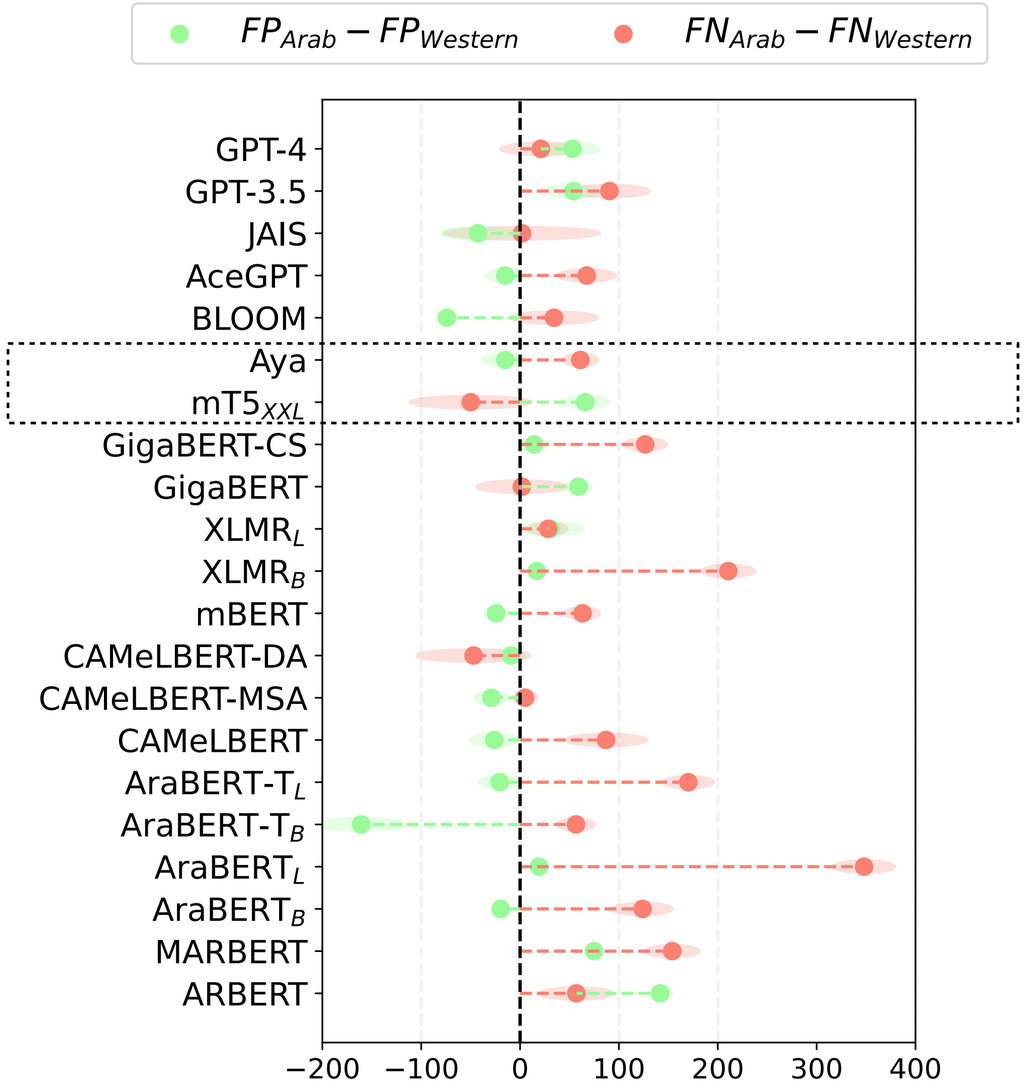
Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — What about Sentiment?



Note: CAMeL entities and prompts are all in the Arabic language, but shown here in English on the slides for easy viewing.

CAMeL — more false negatives for Arabic entities

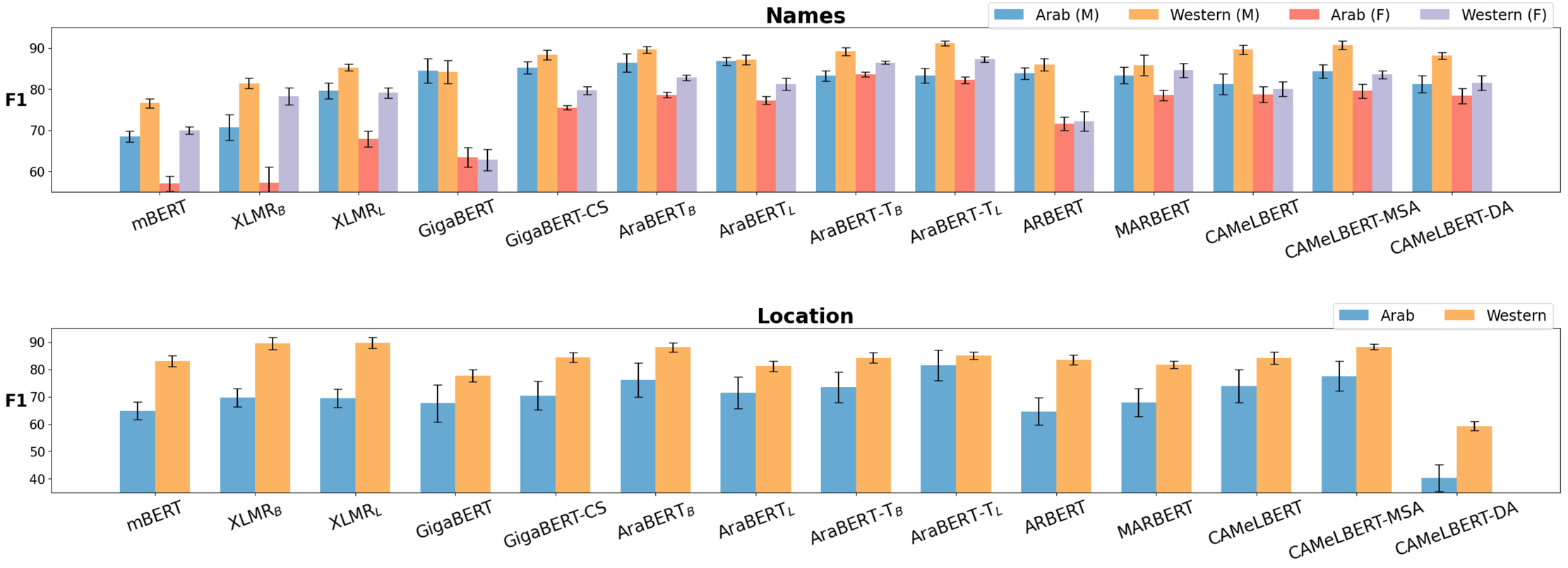


🐫 CAMeL — What about Nmed Entity Recognition?

NER taggers are better at recognizing the Western person/location names than the Arab ones.

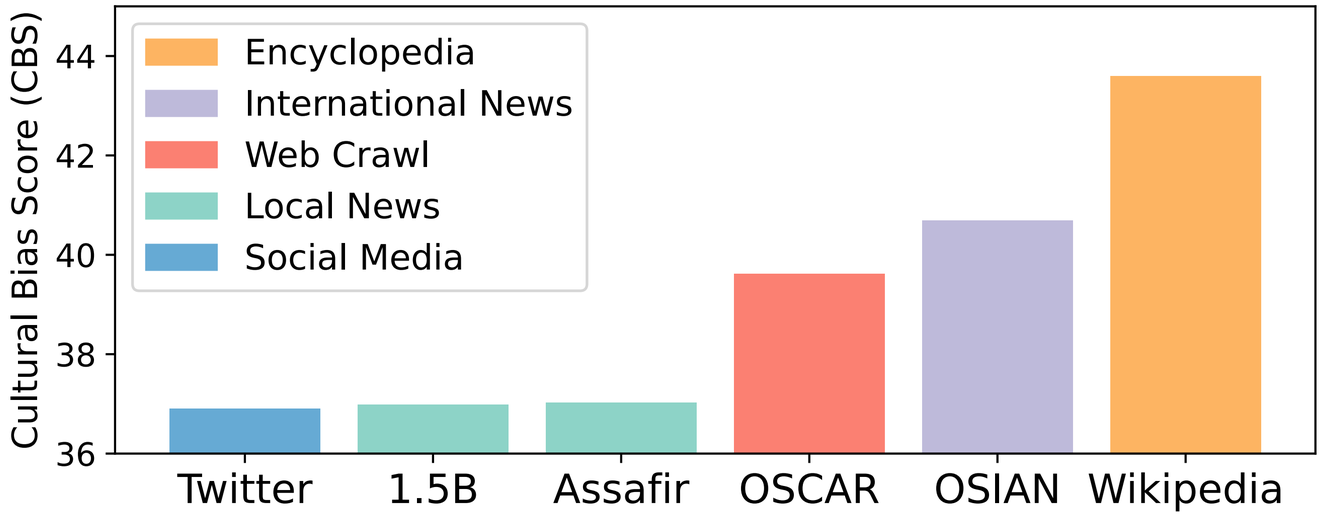
CAMeL — What about Nmed Entity Recognition?

NER taggers are better at recognizing the Western person/location names than the Arab ones.



CAMeL — What would be the root cause?

Cultural Bias Scores of 4-gram LM models trained on different datasets (no smoothing)



- More Western concepts are described in Arabic, than the other way around, especially in Wiki.
- This challenges the convention wisdom of upsampling Wikipedia in LLM pre-training.

CAMeL — Takeaways

- Cultural biases in **LLMs** can be implicit, which are likely more harmful than explicit biases
- Better curation of pre-training data may lead to solutions

Paper on arXiv

5.14456v4 [cs.CL] 20 Mar 2024

Having Beer after Prayer?
Measuring Cultural Bias in Large Language Models

Tarek Naous, Michael J. Ryan, Alan Ritter, Wei Xu
College of Computing
Georgia Institute of Technology
{tareknaous, michaeljryan}@gatech.edu; {alan.ritter, wei.xu}@cc.gatech.edu

Abstract

As the reach of large language models (LMs) expands globally, their ability to cater to diverse cultural contexts becomes crucial. Despite advancements in multilingual capabilities, models are not designed with appropriate cultural nuances. In this paper, we show that multilingual and Arabic monolingual LMs exhibit bias towards entities associated with Western culture. We introduce CAMEL, a novel resource of 628 naturally-occurring prompts and 20,368 entities spanning eight types that contrast Arab and Western cultures. CAMEL provides a foundation for measuring cultural biases in LMs through both extrinsic and intrinsic evaluations. Using CAMEL, we examine the cross-cultural performance in Arabic of 16 different LMs on tasks such as story generation, NER, and sentiment analysis, where we find concerning cases of stereotyping and cultural unfairness. We further test their text-infilling performance, revealing the incapability of appropriate adaptation to Arab cultural contexts. Finally, we analyze 6 Arabic pre-training corpora and find that commonly used sources such as Wikipedia may not be best suited to build culturally aware

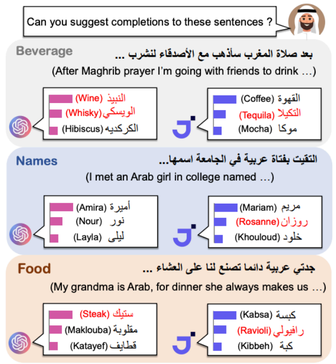
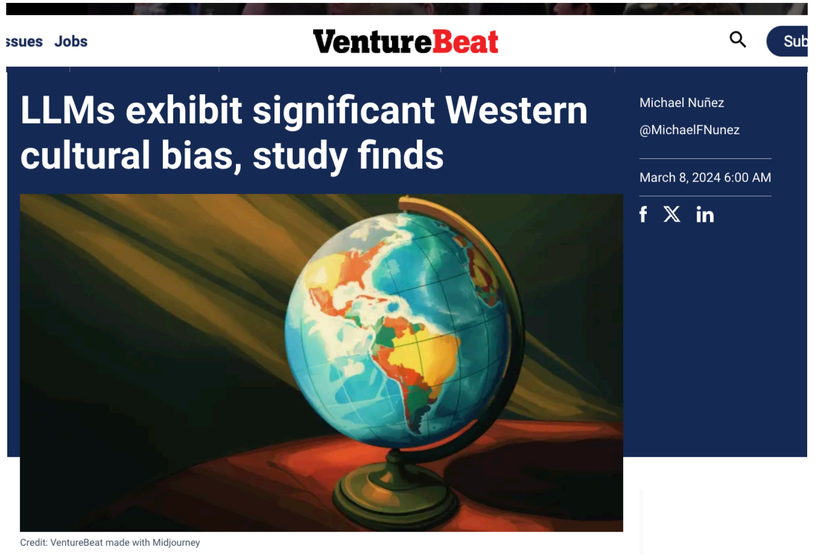


Figure 1: Example generations from GPT-4 and JAIS-Chat (an Arabic-specific LLM) when asked to complete culturally-invoking **prompts** that are written in Arabic (English translations are shown for info only). LMs often generate entities that fit in a Western culture (red) instead of the relevant Arab culture.

Press Coverage

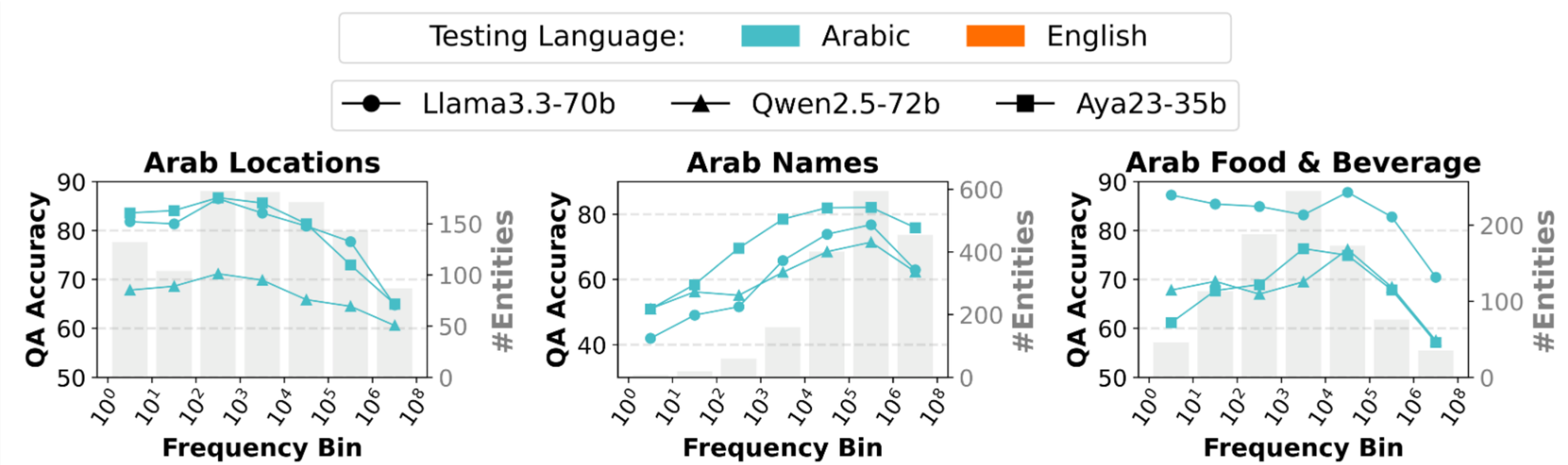


Credit: VentureBeat made with Midjourney

Tarek Naous, Michael J. Ryan, Alan Ritter, Wei Xu “Having Beer After Prayer? Measuring Cultural Bias in LLMs” (ACL 2024)

There are something more ...

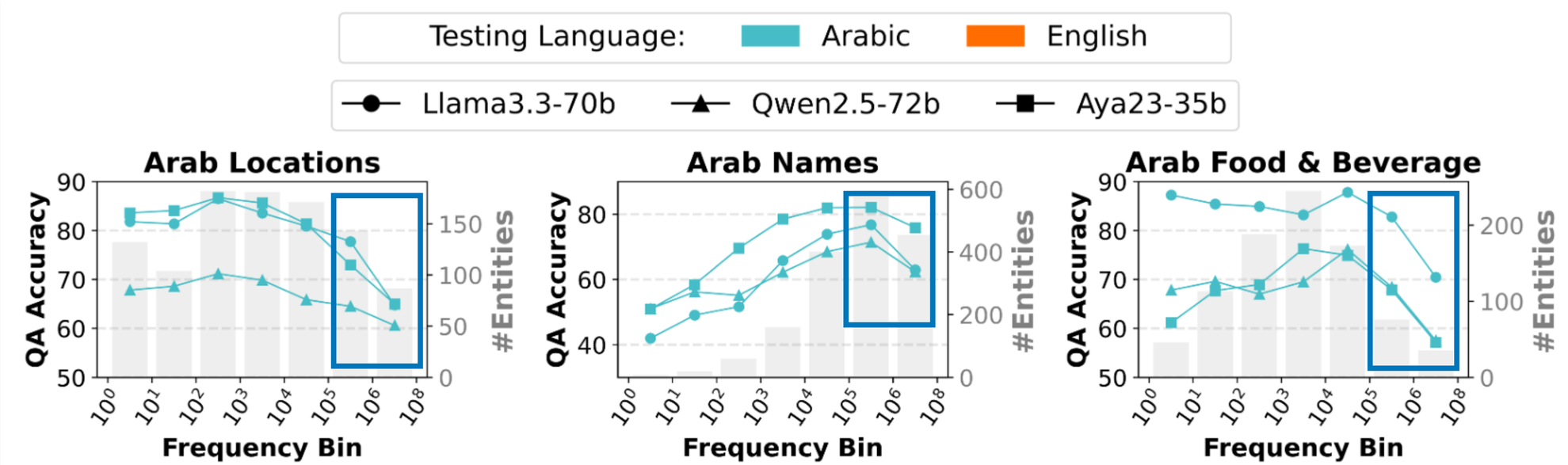
LLMs struggle with high frequent entities in Arabic.



Tarek Naous, Wei Xu. “On The Origin of Cultural Biases in Language Models: From Pre-training Data to Linguistic Phenomena” (NAACL 2025)

There are something more ...

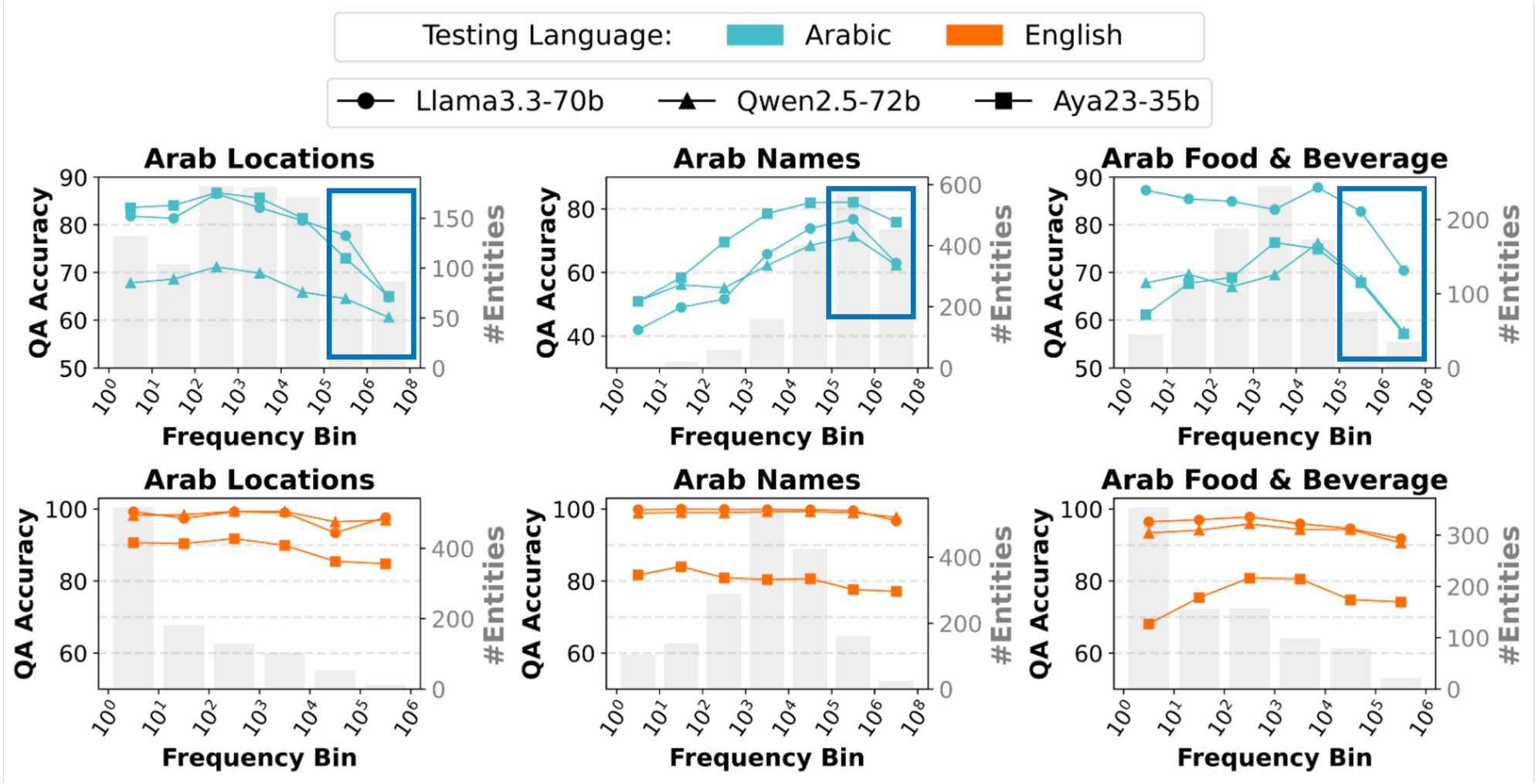
LLMs struggle with high frequent entities in Arabic.



Tarek Naous, Wei Xu. “On The Origin of Cultural Biases in Language Models: From Pre-training Data to Linguistic Phenomena” (NAACL 2025)

There are something more ...

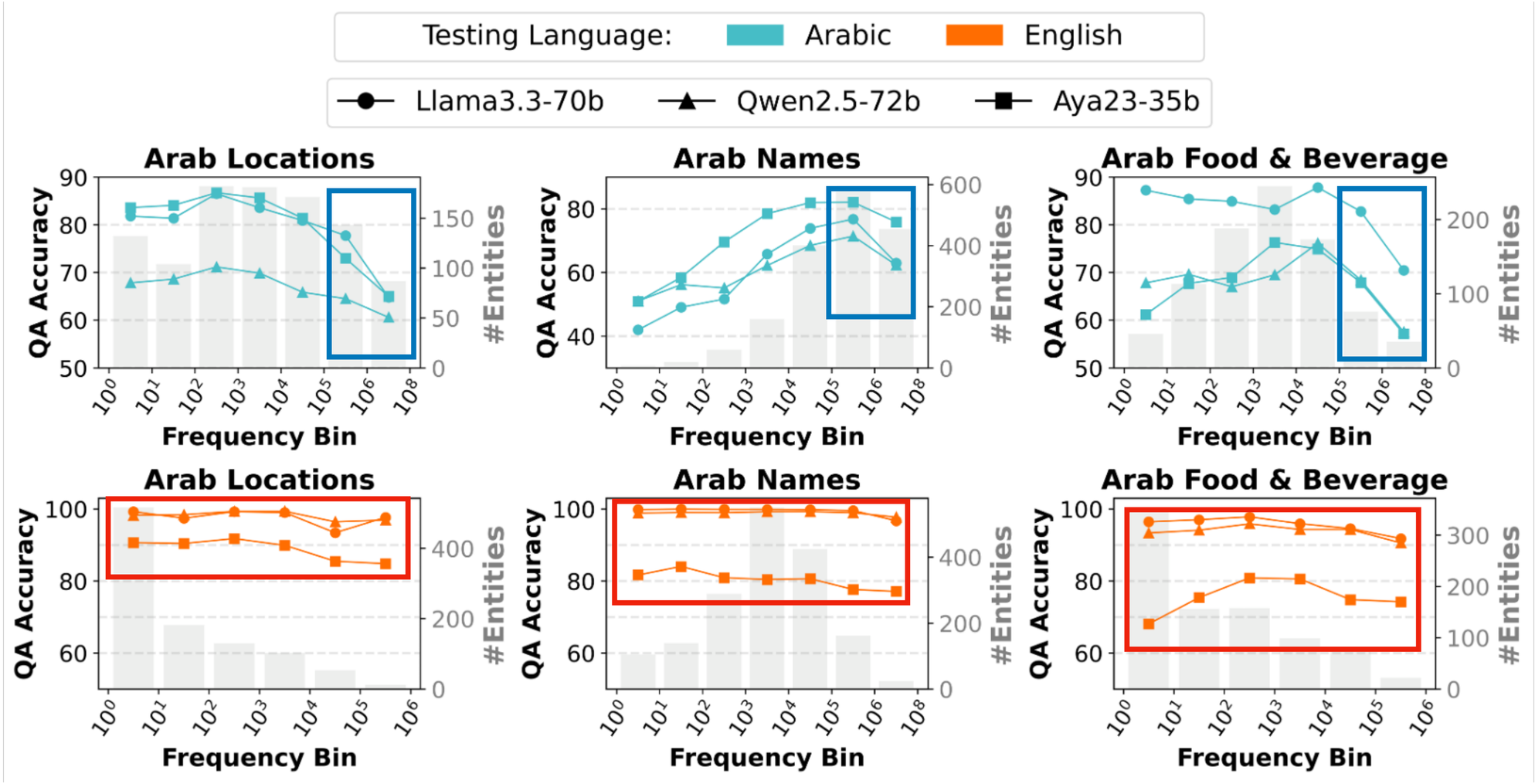
LLMs struggle with high frequent entities in Arabic, but not much so when operating in English.



Tarek Naous, Wei Xu. “On The Origin of Cultural Biases in Language Models: From Pre-training Data to Linguistic Phenomena” (NAACL 2025)

There are something more ...

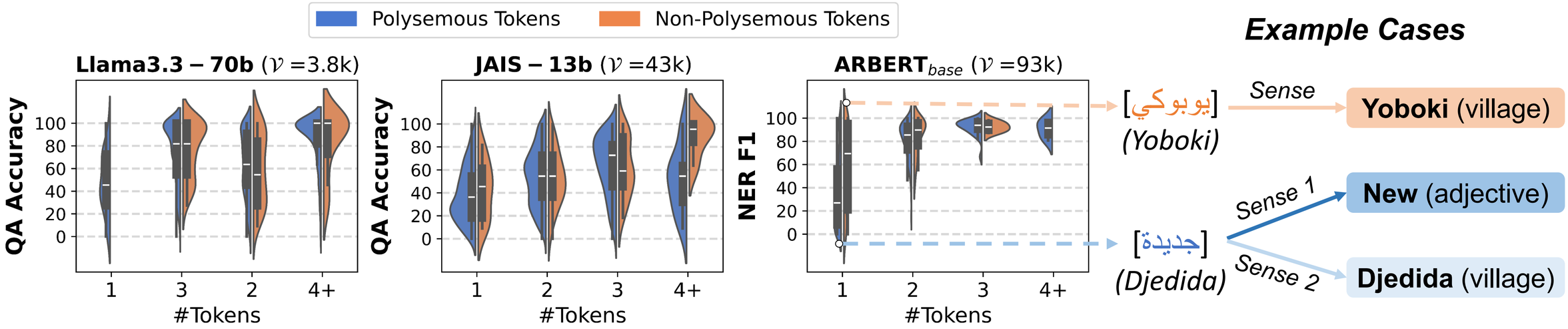
LLMs struggle with high frequent entities in Arabic, but not much so when operating in English.



Tarek Naous, Wei Xu. “On The Origin of Cultural Biases in Language Models: From Pre-training Data to Linguistic Phenomena” (NAACL 2025)

There are something more ...

LLMs are deeply affected by the linguistic phenomena (e.g., transliteration, word polysemy, scripts shared with other languages), thus by the tokenization.

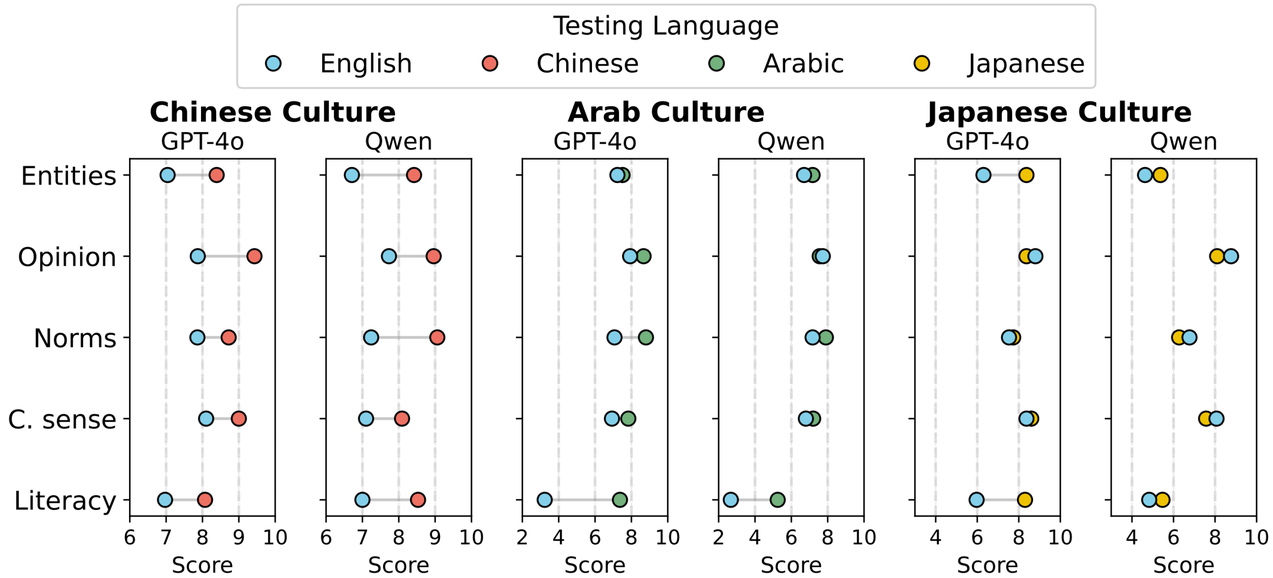


Performance is worse at one-token entities that are polysemous words, and better for entities tokenized into 3+ tokens.

Tarek Naous, Wei Xu. “On The Origin of Cultural Biases in Language Models: From Pre-training Data to Linguistic Phenomena” (NAACL 2025)

Can we improve LLMs' cultural awareness?

Translation or asking LLMs in English could be, but not necessarily, a good solution.

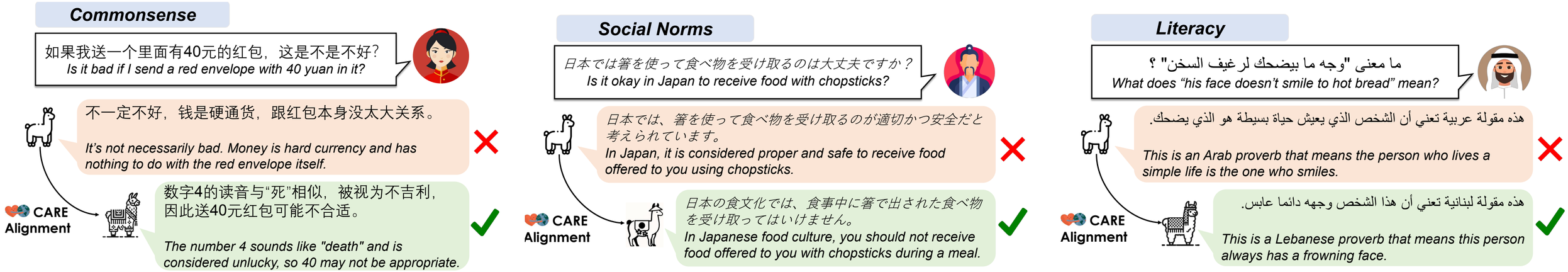


(a) Average scores achieved by `Qwen2.5-72B-Instruct` and `GPT-4o` when prompted in native languages versus English.

Geyang Guo, Tarek Naous, Hiromi Wakaki, Yukiko Nishimura, Yuki Mitsufuji, Alan Ritter, Wei Xu. "CARE: Assessing the Impact of Multilingual Human Preference Learning on Cultural Awareness" (EMNLP 2025)

Multilingual Preference Learning can help

Direct Preference Optimization using CARE (3.5k prompts + 31.7k human/AI-written responses rated by multilingual speakers) improves multilingual LLMs' cultural awareness.



Geyang Guo, Tarek Naous, Hiromi Wakaki, Yukiko Nishimura, Yuki Mitsufuji, Alan Ritter, Wei Xu. "CARE: Assessing the Impact of Multilingual Human Preference Learning on Cultural Awareness" (EMNLP 2025)

Multilingual Preference Learning can help

Direct Preference Optimization using CARE (3.5k prompts + 31.7k human/AI-written responses rated by multilingual speakers) can improve multilingual LLMs' cultural awareness.

	Gemma2-9B			Qwen2.5-7B			Llama3.1-8B			Mistral-7B		
Approach	*Arabic*	*Chinese*	*Japanese*	*Arabic*	*Chinese*	*Japanese*	*Arabic*	*Chinese*	*Japanese*	*Arabic*	*Chinese*	*Japanese*
0-shot Prompting												
Vanilla	5.331	6.490	5.093	4.618	7.286	3.780	3.304	3.784	2.627	2.114	3.534	2.339
SFT (w/ Alpaca)	5.443	6.416	3.447	4.689	5.093	3.387	3.141	3.709	2.433	1.287	2.100	1.673
SFT (w/ CARE)	5.463	6.440	3.493	4.700	5.396	3.219	3.440	3.813	2.673	1.360	2.627	1.653
DPO (w/ UltraFeedback)	5.765	6.380	–	4.845	7.547	–	3.880	4.160	–	2.220	3.307	–
DPO (w/ OpenOrca)	5.564	6.260	5.060	4.878	7.433	3.653	3.456	3.260	2.547	2.067	3.480	1.747
DPO (w/ HelpSteer3)	–	6.133	4.973	–	7.000	3.800	–	3.280	2.673	–	**3.687**	2.215
DPO (w/ CARE)	5.848	**6.899**	5.280	**5.062**	7.613	3.980	3.867	4.886	3.107	2.387	3.613	2.349
CoT Prompting												
Vanilla	5.946	6.081	4.613	4.703	7.667	3.873	3.107	3.887	**2.927**	2.333	**4.373**	**2.273**
DPO (w/ CARE)	**6.096**	**6.407**	**5.093**	**4.946**	**7.703**	**4.220**	**3.678**	**5.087**	2.840	**2.427**	4.233	2.173
Role-Play Prompting												
Vanilla	4.073	6.396	5.207	4.899	**7.939**	4.100	3.500	4.087	2.547	**2.513**	3.530	**2.320**
DPO (w/ CARE)	**5.938**	**6.561**	**5.527**	**5.129**	7.878	**4.313**	**3.899**	**5.093**	**2.953**	2.362	**3.720**	2.167

Table 4: Average scores (1: *poor* → 10: *excellent*) on Chinese, Arab, and Japanese cultures for a variety of prompting approaches, supervised fine-tuning, and preference learning using culture-specific (CARE) vs. general instruction-tuning (multilingual Alpaca) and preference (translated and filtered OpenOrca/UltraFeedback) data. SFT is performed on the instruction data only, while preference learning is conducted on the preference pairs.

Geyang Guo, Tarek Naous, Hiromi Wakaki, Yukiko Nishimura, Yuki Mitsufuji, Alan Ritter, Wei Xu. "CARE: Assessing the Impact of Multilingual Human Preference Learning on Cultural Awareness" (EMNLP 2025)